Agent-Notary: Cryptographic Non-Repudiation, Zero-Knowledge Invariant Attestation, and Merkle-DAG Action Provenance for Autonomous Multi-Agent Swarms

Aniket Agrawal
Cloud Engineer, Google Cloud
Google, Bangalore, India
aniketagrawal@google.com

***Abstract*—As enterprise computing systems adopt multi-agent workflows—coordinated across platforms such as Google Cloud Vertex AI Agent Builder and the Model Context Protocol (MCP)—agents increasingly execute state-mutating operations across external databases, financial settlement APIs, and cloud infrastructure. However, existing agent orchestration runtimes lack cryptographic action provenance and non-repudiation across service boundaries. When a sub-agent executes a tool invocation induced by an indirect prompt injection or reasoning error, standard telemetry systems (e.g., OpenTelemetry, cloud audit trails) record only mutable, unauthenticated textual logs that can be modified, repudiated, or omitted by compromised components. Furthermore, enterprise compliance requires cryptographic verification that policy invariants were satisfied without exposing sensitive customer context or raw prompt tokens to external auditors.**

In this paper, we present Agent-Notary, a low-latency cryptographic attestation and non-repudiation framework for distributed multi-agent workflows. Agent-Notary introduces three core mechanisms: (1) an append-only Causal Merkle-DAG that binds multi-agent request and completion vertices ($v_i^{\text{req}} \rightarrow v_i^{\text{resp}}$), tool calls, and Lamport timestamps into an immutable cryptographic chain of custody without retroactive digest mutation; (2) a Dual-Tier Blinded Policy-Invariant Attestation Engine that pairs microsecond intra-cluster HKDF-blinded enclave witness tokens on the hot path with hardware-attested Confidential VM commitment equivalence and asynchronous Ristretto255 Pedersen commitments with Fiat-Shamir Bulletproof range/predicate transcripts for third-party public auditability without disclosing sensitive parameters; and (3) a wire-level Zero-Knowledge Action Attestation Voucher (ZKA-Voucher) encapsulated into standard HTTP/2 and gRPC transport metadata headers. Agent-Notary implements a dual-mode cryptographic trust model: service-bound symmetric HMAC authenticators for microsecond intra-cluster tool server verification alongside asymmetric Ed25519 signatures on Merkle frontier epoch roots for third-party public non-repudiation. Empirical evaluation over $N = 1{,}000$ wire/replay tamper traces and $N = 300$ multi-agent indirect prompt-injection traces (modeled on AgentDojo, InjecAgent, and ASB) demonstrates 100.00% detection of wire tampering, signature forgery, replay desynchronization, and out-of-policy prompt injections ($\Pi(t_i, \theta_i) = 0$), 100.00% $(100/100)$ post-hoc cryptographic parent-slice attribution of in-policy semantic hijacking ($1.84\,\mu$s DAG traversal isolating 82% $[82/82]$ context-isolated traces structurally to a single parent, and narrowing the 18% $[18/18]$ merged-turn subset to the exact k_p-element authenticated parent slice, within which heuristic AST matching pinpoints un-obfuscated parent blobs in $+14.2\,\mu$s), a line-rate mean intra-cluster verification latency of $4.17\,\mu$s (P50: $2.29\,\mu$s, P95: $11.46\,\mu$s, P99: $21.96\,\mu$s), and a compact wire header footprint of $2{,}250$ bytes.

***Index Terms*—Multi-Agent Systems, Non-Repudiation, Zero-Knowledge Proofs, Merkle DAG, Model Context Protocol, Cryptographic Audit, Cloud Security.**

I. Introduction

Large Language Model (LLM) agents are increasingly deployed as distributed multi-agent workflows that interact directly with enterprise backends [1]–[3]. Within enterprise platforms like Vertex AI Agent Builder and the open Model Context Protocol (MCP) [4], agent workflows coordinate multi-step operations including enterprise resource planning (ERP) reconciliations, cloud infrastructure provisioning, and financial disbursements.

However, enterprise multi-agent deployments lack end-to-end cryptographic provenance and non-repudiation across delegation hops [5]–[7]. In a multi-turn workflow, Agent A (e.g., an external customer service intake agent) may ingest an adversarial ticket containing an indirect prompt injection. That injection induces Agent A to issue a compromised task delegation to Agent B (e.g., a privileged financial payment agent), which in turn executes a high-value disbursement API call. When forensic auditors inspect the incident post-hoc, existing telemetry frameworks such as OpenTelemetry and centralized database audit tables provide only mutable, unstructured JSON text logs. These logs exhibit three security limitations:

1) **Vulnerability to Repudiation and Forgery:** Standard API logs lack cryptographic signatures tied to causal execution states. A rogue internal service or an attacker with administrative cloud access can forge, modify, or delete log entries without detection.
2) **Causal Disconnection in Heterogeneous Swarms:** When agents operate across organizational or tenant boundaries, asynchronous tool calls become causally untraceable. Standard tracing spans cannot cryptographically prove which upstream agent instruction induced a downstream mutation.

3) **Auditability Under Confidentiality Constraints:** Regulatory bodies (e.g., SOX, HIPAA, GDPR) require verification that agent tool invocations adhered to enterprise policy invariants (e.g., transaction limits, sanitized destination buckets). However, providing raw audit logs to external auditors exposes proprietary system prompts, model reasoning chains, and confidential customer PII [8].

To overcome these challenges, this paper presents **Agent-Notary**, a line-rate cryptographic non-repudiation framework designed for enterprise multi-agent tool execution. Importantly, Agent-Notary establishes a *two-pronged defense posture* (pre-execution enforcement and post-hoc provenance) under an explicit **Trusted Computing Base (TCB)**: the out-of-process Agent-Notary Gateway sidecar and the host orchestration harness's deterministic sub-session context-propagation multiplexer execute co-resident inside the calling agent's hardware-attested Confidential VM enclave boundary (backed by AMD SEV-SNP memory encryption), while destination tool-server verifiers execute inside their respective remote attested enclave boundaries across untrusted network links alongside administrator-provisioned declarative policy circuits Π. Conversely, LLM reasoning models, host OS/hypervisor/network/logging administrators outside the Confidential VMs, inter-agent network links, local observability databases, and external third-party auditors are strictly untrusted; Google Cloud KMS key release is cryptographically gated by SEV-SNP attestation quotes ($\mathcal{Q}_{\text{TEE}}$), Cloud Spanner/Storage epoch audit anchors enforce WORM retention locks (and replicate Σ_{root} to an external auditor domain), and the gateway journals tool responses to a write-once AES-256-GCM encrypted payload escrow store. Specifically, a per-epoch root escrow secret $\text{sk}_{\text{escrow}}$ is wrapped once per epoch under the KMS-attested auditor public key, and each vertex derives a single-use key $k_{\text{escrow},k} = \text{HKDF-SHA256}(\text{sk}_{\text{escrow}}, \texttt{0x11} \parallel \texttt{0x01} \parallel H(v_k^{\text{resp}}))$ and deterministic 96-bit (12-octet) initialization vector $\text{IV}_k = \text{HKDF-Expand}_{96}(\text{HKDF-Extract}(\text{salt}_{\text{notary}}, \text{sk}_{\text{escrow}}), \texttt{0x11} \parallel \texttt{0x02} \parallel H(v_k^{\text{resp}}))$—where, as formalized in Section IV-A, two-argument $\text{HKDF-SHA256}(\text{ikm}, \text{info})$ shorthand expands from the same extracted pseudorandom key $\text{PRK}_{\text{escrow}} = \text{HKDF-Extract}(\text{salt}_{\text{notary}}, \text{sk}_{\text{escrow}})$ so the auditor unwraps $\text{sk}_{\text{escrow}}$ once per epoch and derives $(k_{\text{escrow},k}, \text{IV}_k)$ in memory without per-vertex asymmetric overhead—to encrypt the deterministic RFC 8785 canonical byte buffer $\text{CanonicalJSON}(o_k)$ under AES-256-GCM with $H(v_k^{\text{resp}})$ bound as the Additional Authenticated Data (AAD), guaranteeing deterministic ciphertext idempotency across write retries and immediate AEAD tag rejection if an adversary substitutes ciphertexts across vertices. Crucially, in the single-parent regime ($|\mathcal{P}_i^{\text{req}}| = 1$, 82% of traces), the enclave-resident harness multiplexer spawns per-branch context-isolated LLM sub-invocations whose outbound transport channels are bound to transport-layer session handles mapped to v_k^{resp} (without relying on the untrusted LLM to echo proxy tokens), whereas multi-parent context merging ($|\mathcal{P}_i^{\text{req}}| = k_p > 1$, 18% of traces) is governed strictly by deterministic join barriers in the trusted harness workflow graph. If a privileged cloud administrator attempts to suppress or truncate already-anchored gateway frames from audit storage, monotonic epoch sequence counters ($\text{seq}_{\text{epoch}}$) and causal parent links in anchored frontier roots expose the omission immediately; while non-escrowed (`escrow_mode` $= 0$) calls trade off a bounded $\Delta_{\text{epoch}} = 100\,\text{ms}$ pre-anchoring window for sub-5 μs dispatch, production deployments enforce a deterministic three-tier policy matrix mapping (i) read-only/idempotent queries to single-node `escrow_mode` $= 0$ ($28.45\,\mu$s, where the destination verifier's append-only receipt log still records $\mathcal{V}_{\text{wire}}$), (ii) intermediate-risk reversible mutations to `escrow_mode` $= 0$ paired with synchronous multi-replica Redis/`WAIT 1` and regional Persistent Disk journaling ($213.45\,\mu$s, eliminating single-host crash loss prior to the $100\,\text{ms}$ tick), and (iii) high-value or irreversible external mutations to `escrow_mode` $= 1$ ($58.40\,\text{ms}$) so destination verifiers hold execution in provisional escrow until Spanner epoch anchoring completes. Conversely, if an unnotarized request is dropped before reaching the gateway sidecar, no ZKA-Voucher is minted and the destination verifier fails closed; if a notarized request ($\mathcal{V}_{\text{wire}}$) is dropped in transit before reaching the destination tool server or if the returning completion frame (o_i) is dropped after tool execution, the pre-execution vertex v_i^{req} (H_{pending}) remains permanently anchored in R_T as an unclosed invocation, and the destination verifier's append-only execution receipt log indexed by $(H_{\text{leaf}}, \text{vid})$ resolves retries idempotently: each destination verifier authenticates execution/error receipts ($\texttt{0x09} \parallel \texttt{0x01} \parallel H_{\text{leaf}} \parallel \text{lp}(\text{vid}) \parallel H_{\text{out}}(v_i^{\text{resp}}) \parallel \text{encode}_8(\text{status}) \parallel \text{encode}_{64}(\tau_{\text{ver}})$) and `NOT_SEEN` status inquiry receipts ($\texttt{0x09} \parallel \texttt{0x02} \parallel H_{\text{leaf}} \parallel \text{lp}(\text{vid}) \parallel$ "NOT_SEEN" $\parallel \text{encode}_{64}(\tau_{\text{ver}})$) using its enclave Ed25519 signing key $\text{sk}_{\text{verifier},t_i}$ (whose public key $\text{pk}_{\text{verifier},t_i}$ is bound in the verifier's SEV-SNP attestation quote $\mathcal{Q}_{\text{TEE}}$ for third-party public audit verification without K_{tool,t_i}, paired with intra-cluster $\text{HMAC-SHA256}_{K_{\text{tool},t_i}}$ for microsecond gateway checks). Upon a transport timeout, the gateway always replays the original $\mathcal{V}_{\text{wire}}$ carrying $(H_{\text{leaf}}, \text{vid})$ first—if the initial request executed before the response o_i was dropped, the verifier's Step 2 replay check looks up $(H_{\text{leaf}}, \text{vid})$ in the receipt log and returns the cached signed execution receipt and output o_i idempotently without re-executing t_i, whereas if the initial frame never reached the verifier (or only after an explicit $\text{sk}_{\text{verifier},t_i}$-signed $\texttt{0x09} \parallel \texttt{0x02}$ `NOT_SEEN` status inquiry), the replayed $\mathcal{V}_{\text{wire}}$ executes normally (or a fresh retry vertex vid' explicitly links to the unclosed v_i^{req}). Under this threat model, Agent-Notary enforces: (i) *pre-execution line-rate enforcement*, deterministically blocking tool invocations whose parameters violate policy invariants ($\Pi(t_i, \theta_i) = 0$) or exhibit wire tampering, forgery, or replay; and (ii) *post-hoc non-repudiable causal provenance*, anchoring multi-agent delegation chains in an append-only Causal Merkle-DAG so that in-policy semantic prompt injections ($\Pi(t_i, \theta_i) = 1$) remain forensically attributable across organizational boundaries without repudiation via the **Zero-Knowledge Action Attestation Voucher (ZKA-Voucher)**.

II. Related Work

A. Multi-Agent Systems, Request Signing, and Privilege Control

The emergence of standardized tool invocation protocols and benchmarks—most notably the Model Context Protocol (MCP) [4], functional suites such as MCP-Universe [9] and MCP-Bench [10], and dedicated MCP security benchmarks including MCPSecBench [11], MCPTox [12], and MCP-SafetyBench [13]—alongside HTTP Message Signatures (RFC 9421) [14], OAuth 2.0 Token Exchange (RFC 8693) [15], Demonstrating Proof-of-Possession (DPoP, RFC 9449) [16], and attenuated capability token architectures such as Macaroons [17] and public-key attenuated Eclipse Biscuit tokens [18], has advanced how distributed services authenticate remote operations. Recent agent identity and governance architectures such as BAID [19], AIP [20], SAGA [21], and workflow-bound Agentic JWT (A-JWT) delegation extensions over RFC 7519/8693/9449 [15], [16], [22], [51] further introduce cryptographic agent identifiers, lifecycle access-control gateways, and session tokens across MCP and A2A boundaries. However, existing authentication and capability frameworks treat authorization predominantly as a session-level handshake or linear delegation chain [23]. Once an agent establishes an authenticated session, the runtime often retains broad authority across concurrent branches [24], [25]. While Macaroons provide chained HMAC caveats, Eclipse Biscuit [18], AIP [20], SAGA [21], and A-JWT [22] add public-key attenuation, governance registries, or workflow token claims, and RFC 9421 / DPoP specify per-request HTTP signing, none bind non-linear branching two-phase causal dependency DAGs ($v_i^{\text{req}} \to v_i^{\text{resp}}$) across asynchronous multi-agent swarms nor provide zero-knowledge policy-invariant attestations over blinded tool arguments.

Concurrently, recent agent-layer security and trace-verification frameworks—including CaMeL [26], FIDES [27], Progent [28], and Verifiable Execution Traces (VET) [29]—enforce pre-execution data-flow isolation, capability labels, deterministic tool policies, or step-level execution trace signatures. While CaMeL, FIDES, and Progent effectively restrict unauthorized tool calls within a single trusted runtime and VET signs linear step traces, they do not generate sub-5 μs service-scoped wire vouchers, non-repudiable cross-domain two-phase Merkle-DAG frontier anchors with WORM payload escrow, or Dual-Tier Ristretto255 Bulletproof R1CS parameter proofs verifiable by external third-party auditors when logs are contested or local telemetry is compromised. Agent-Notary is directly complementary to intra-agent IFC/policy engines, providing the cryptographic transport and non-repudiation layer across distributed multi-agent boundaries.

B. Cryptographic Provenance and Transparency Systems

Cryptographic transparency and supply-chain provenance systems have proven effective in securing distributed infrastructure, most prominently Certificate Transparency (RFC 6962) [30], software supply-chain layout attestation in *in-toto* [31], and content-addressed Merkle-CRDTs [32]. RFC 6962 employs append-only Merkle trees with domain-separated leaf and interior prefixes to detect unauthorized TLS certificate issuance, *in-toto* verifies signed multi-step software pipeline layouts, and Merkle-CRDTs embed causal DAG history into self-verifying conflict-free replicated data types. Similarly, recent W3C PROV-based agentic provenance trackers such as PROV-AGENT [33] capture semantic causal graphs across MCP tool executions, while emerging COSE-based supply-chain transparency standards such as IETF SCITT [34] (building on RFC 6962 [30]) issue signed Merkle inclusion receipts over registered artifacts. However, W3C PROV-AGENT graphs are unauthenticated telemetry records lacking cryptographic non-repudiation or zero-knowledge privacy; classical transparency logs and IETF SCITT receipts target linear artifact registration with asymmetric COSE verification rather than sub-5 μs branching multi-agent RPC streams; *in-toto* targets static build pipelines; and Merkle-CRDTs optimize for eventual-consistency state replication rather than synchronous two-phase request-completion causality ($v_i^{\text{req}} \to v_i^{\text{resp}}$) with blinded zero-knowledge policy-invariant proofs [35].

C. Zero-Knowledge and Blinded Policy Attestation

Zero-Knowledge Proofs (ZKPs) allow a prover to convince a verifier that an assertion is mathematically true without disclosing the underlying witness data [8], [36]. While general-purpose zk-SNARKs and zk-STARKs have been explored for verifiable off-chain computation [37], recent MCP auditing proposals such as zk-MCP [38] apply zero-knowledge circuit proofs to agent communication logs, and Bulletproofs [39] provide short non-interactive range proofs over Pedersen commitments [40] without a trusted setup, evaluating multi-millisecond elliptic-curve or SNARK proofs synchronously on every intra-cluster RPC would bottleneck high-frequency agent loops. Agent-Notary bridges this gap through a Dual-Tier Blinded Policy-Invariant Attestation Engine: deploying service-bound HKDF-blinded symmetric enclave commitments on the sub-5 μs intra-cluster hot path while deferring elliptic-curve Ristretto255 Pedersen commitments and Fiat-Shamir Bulletproof range and R1CS set-membership transcripts [39], [41] anchored under hardware-attested Ed25519 epoch signatures entirely to asynchronous frontier epoch anchoring for third-party public auditability.

III. Formal Problem Formulation

Let a multi-agent swarm execution be defined as a set of interacting agents $\mathcal{A} = \{a_1, a_2, \ldots, a_k\}$ invoking a catalog of external enterprise tools $\mathcal{T} = \{t_1, t_2, \ldots, t_m\}$.

Definition 1 (Policy Invariant)**.** *Let $\Theta \triangleq \bigcup_{t \in \mathcal{T}} \Theta_t$ denote the universe of tool parameter spaces, where Θ_t is the schema domain of tool t. A policy invariant Π is a deterministic total predicate:*

$$\Pi : \mathcal{T} \times \Theta \to \{0, 1\}, \quad \textit{with } \Pi(t, \theta) = 0 \ \ \forall \theta \notin \Theta_t \tag{1}$$

An execution is policy-compliant if and only if $\theta \in \Theta_t$ and $\Pi(t, \theta) = 1$; any tool call with $\Pi(t, \theta) = 0$ is rejected pre-execution at the gateway prover boundary.

Definition 2 (Append-Only Two-Phase Causal Merkle-DAG Lifecycle)**.** *Each action step i in the swarm is represented in an append-only directed acyclic graph $\mathcal{G}_{prov} = (\mathcal{V}, \mathcal{E})$ via two distinct, causally linked immutable vertices ($v_i^{req}, v_i^{resp} \in \mathcal{V}$ with globally unique identifiers id_v and directed edge $(v_i^{req}, v_i^{resp}) \in \mathcal{E}$), serialized under deterministic RFC 8785 JSON Canonicalization Scheme (JCS) [42] (*$\text{CanonicalJSON}(\cdot)$*) so that post-execution output binding never mutates or retroactively invalidates pre-execution vertex digests. Even when high-level agent workflows contain cyclic iterative delegation loops (e.g., planner-critic refinement $a_2 \leftrightarrow a_3$), the enclave harness multiplexer unrolls each loop iteration into a fresh monotonically timestamped vertex pair $v_k^{req} \rightarrow v_k^{resp}$ with a fresh UUID id_k and strictly increasing Lamport clock λ_k, guaranteeing that $\mathcal{G}_{prov}$ remains a finite acyclic DAG. To guarantee unconditional prefix-level domain separation across all eight Merkle-DAG SHA-256 contexts (and across all 19 top-level SHA-256/Ed25519/HKDF/HMAC domain tags* `0x00`–`0x12` *in the protocol—comprising 24 disjoint cryptographic contexts when counting fixed-length 2-byte sub-tags* `0x09` $\|$ `0x00`–`0x02`, `0x0F` $\|$ `0x01`–`0x02`, `0x11` $\|$ `0x01`–`0x02`, *and* `0x12` $\|$ `0x01`–`0x02`*), 1-byte tags* `0x00` *and* `0x01` *are reserved exclusively for RFC 6962 Merkle tree leaf and interior nodes, while vertex-level commitments use tags* `0x02` *through* `0x07` *(and signature/attestation/escrow/synchronization contexts use* `0x08` *through* `0x12`*):*

1) ***Pre-Execution Request Vertex (v_i^{req}):*** *Minted before dispatching tool t_i, committing to parameters via a cryptographically random blinding salt $r_i \xleftarrow{\$} \{0,1\}^{256}$ with 1-byte domain separator* `0x02`*:*

$$C_{\theta_i} = \text{Commit}(\theta_i; r_i) \triangleq \text{SHA256}\left(\texttt{0x02} \parallel r_i \parallel \text{CanonicalJSON}(\theta_i)\right) \tag{2}$$

The immutable request vertex tuple is defined as:

$$v_i^{req} = \langle id_i^{req}, a_i, \lambda_i, \mathcal{P}_i^{req}, t_i, C_{\theta_i}, H_{out}(v_i^{req}), \pi_i \rangle \tag{3}$$

where a_i is the calling agent, $\lambda_i \in \mathbb{N}$ is the causal Lamport clock at dispatch, $\mathcal{P}_i^{req} = \{v_p \mid (v_p, v_i^{req}) \in \mathcal{E}\}$ is the set of causal parent vertices active in the agent's subsession context at dispatch time, $H_{out}(v_i^{req}) \triangleq H_{pending} \triangleq \text{SHA256}(\texttt{0x06} \parallel$ "PENDING_EXECUTION"), and π_i is the policy-invariant attestation token. The digest $H_{leaf} \triangleq H(v_i^{req})$ is permanently embedded in the wire ZKA-Voucher $\mathcal{V}_{wire}$.*

2) ***Post-Execution Completion Vertex (v_i^{resp}):*** *Upon tool completion returning payload o_i, rather than mutating v_i^{req} in place, the runtime advances its current local Lamport clock λ_{local} to $\lambda_i^{resp} = \max(\lambda_{local}, \lambda_i) + 1$ and appends a new immutable completion vertex:*

$$v_i^{resp} = \langle id_i^{resp}, a_i, \lambda_i^{resp}, \{v_i^{req}\}, t_i, C_{\theta_i}, H_{out}(v_i^{resp}), \pi_i \rangle \tag{4}$$

with causal parent $\mathcal{P}_i^{resp} = \{v_i^{req}\}$ and status-tagged output digest $H_{out}(v_i^{resp}) \triangleq \text{SHA256}(\texttt{0x07} \parallel \text{CanonicalJSON}(o_i))$, whose `0x07` *prefix guarantees unconditional domain separation from $H_{pending}$ (*`0x06`*) and RFC 6962 interior nodes (*`0x01`*) regardless of payload byte length. Any downstream agent step j causally conditioned on the output o_i includes $v_i^{resp} \in \mathcal{P}_j^{req}$ (whereas concurrent child calls dispatched prior to t_i's return reference v_i^{req}), preserving strict append-only immutability across all historical frontier roots and in-flight vouchers.*

To prevent algebraic self-cancellation and generalized birthday attacks inherent to bitwise XOR accumulators, parent node commitments for any vertex $v \in \mathcal{V}$ with parent set $\mathcal{P}(v) = \{v_{p_1}, \ldots, v_{p_{k_p}}\}$ of cardinality $k_p = |\mathcal{P}(v)|$ are combined via 64-bit big-endian length-prefixed, domain-separated (`0x03`) concatenation over canonically sorted 32-byte digests:

$$H(\mathcal{P}(v)) \triangleq \text{SHA256}\left(\texttt{0x03} \parallel \text{encode}_{64}(k_p) \parallel H(v_{p_1}) \parallel \cdots \parallel H(v_{p_{k_p}})\right) \tag{5}$$

where $H(v_{p_1}) \leq \cdots \leq H(v_{p_{k_p}})$ lexicographically. For every variable-length string field $s \in \{\text{id}_v, a_i, t_i\}$, let $\text{lp}(s) \triangleq \text{encode}_{64}(|s|) \parallel s$ denote its 64-bit big-endian byte-length prefix followed by its UTF-8 bytes, while $H(\mathcal{P}(v)), C_{\theta_i}, H_{\text{out}}(v), \pi_i \in \{0,1\}^{256}$ are fixed-width 32-byte digests. The canonical vertex commitment $H(v)$ for $v \in \{v_i^{\text{req}}, v_i^{\text{resp}}\}$ is computed with domain separator `0x04`:

$$H(v) = \text{SHA256}\Big(\texttt{0x04} \parallel \text{lp}(\text{id}_v) \parallel \text{lp}(a_i) \parallel \text{encode}_{64}(\lambda_v) \parallel H(\mathcal{P}(v)) \parallel \text{lp}(t_i) \parallel C_{\theta_i} \parallel H_{\text{out}}(v) \parallel \pi_i\Big) \tag{6}$$

At any snapshot T, the active DAG frontier (partitioned per `session_id` shard) is defined strictly as the set of sink vertices in active sessions with zero outgoing causal edges, $\mathcal{V}_{\text{frontier}}(T) \triangleq \{v \in \mathcal{V} \mid \text{out-deg}_{\mathcal{G}_{\text{prov}}}(v) = 0\}$. We distinguish graph-structural supersession from cache eviction: when a child vertex (such as v_i^{resp}) is appended to v_i^{req}, $\text{out-deg}(v_i^{\text{req}}) > 0$ so v_i^{req} naturally transitions out of $\mathcal{V}_{\text{frontier}}$; however, true terminal sink vertices ($\text{out-deg}(v) = 0$) are never evicted by TTL or cache pressure prior to epoch anchoring at T_{epoch}. Once a workflow session closes and its terminal sinks are durably anchored in $R_{T_{\text{epoch}}}$, those completed session sinks are checkpointed into the epoch log and compacted from the active in-memory frontier, keeping active per-session frontier depth bounded ($\lceil \log_2 |\mathcal{V}_{\text{frontier}}| \rceil \in \{3,4,5\}$). Any historical vertex $v_k \notin \mathcal{V}_{\text{frontier}}(T_{\text{epoch}})$ is verified against anchored epoch root $R_{T_{\text{epoch}}}$ by composing its directed causal path $v_k = u_0 \rightarrow u_1 \rightarrow \cdots \rightarrow u_r = v_f \in \mathcal{V}_{\text{frontier}}(T_{\text{epoch}})$ with an auxiliary causal-path witness tuple at each hop $u_j \rightarrow u_{j+1}$—comprising the non-private tuple fields of u_{j+1} (Eq. 6), the sibling co-parent digests $\{H(w) \mid w \in \mathcal{P}(u_{j+1}) \setminus \{u_j\}\}$ at multi-parent merges (Eq. 5), and v_f's epoch inclusion path $\mathcal{M}_{\text{path}}(T_{\text{epoch}})$. To fold $n = |\mathcal{V}_{\text{frontier}}|$ sorted digests $D_{1..n} = \langle H(v_1), \ldots, H(v_n) \rangle$ (with $H(v_1) \leq \cdots \leq H(v_n)$) into a canonical scalar root R_T, the gateway applies exact RFC 6962 Section 2.1 top-down recursive power-of-two splitting [30]: for any single-element slice $D_{j..j}$, $H_{\text{tree}}(D_{j..j}) \triangleq \text{SHA256}(\texttt{0x00} \parallel D_j)$, and

for any slice $D_{1..n}$ of length $n > 1$ with $k = 2^{\lfloor \log_2(n-1) \rfloor}$ (the largest power of two strictly less than n), $H_{\text{tree}}(D_{1..n}) \triangleq \text{SHA256}\left(\texttt{0x01} \parallel H_{\text{tree}}(D_{1..k}) \parallel H_{\text{tree}}(D_{k+1..n})\right)$. Defining $H_{\text{tree}}(\mathcal{V}_{\text{frontier}}) \triangleq H_{\text{tree}}(D_{1..n})$, the top-level cardinality-binding envelope with domain prefix `0x05` is:

$$R_T \triangleq \text{SHA256}\left(\texttt{0x05} \parallel \text{encode}_{64}\left(|\mathcal{V}_{\text{frontier}}|\right) \parallel H_{\text{tree}}(\mathcal{V}_{\text{frontier}})\right) \tag{7}$$

Theorem 1 (Causal Tamper Non-Interference)**.** *Let $\mathcal{G}_{prov} = (\mathcal{V}, \mathcal{E})$ be an append-only Merkle-DAG with globally unique vertex IDs id_v, prefix-free length-prefixed serialization, and active frontier sink vertices $\mathcal{V}_{frontier} = \{v \in \mathcal{V} \mid \text{out-deg}_{\mathcal{G}_{prov}}(v) = 0\}$ committed into scalar root R_T via Eq. (7). Let reflexive-transitive precedence $\preceq$ be defined such that $u \preceq w \iff (u, w) \in \mathcal{E}^*$. For any vertex $v_k \in \mathcal{V}$, there exists at least one frontier sink $v_f \in \mathcal{V}_{frontier}$ such that $v_k \preceq v_f$. If an adversary (i) mutates any historical parameter/salt pair $(\theta_k, r_k) \to (\theta'_k, r'_k) \neq (\theta_k, r_k)$ on $v_c = v_k^{req}$, mutates any output payload $o_k \to o'_k \neq o_k$ on $v_c = v_k^{resp}$, or modifies the parent set $\mathcal{P}(v_c)$ of a surviving child vertex v_c (via causal edge deletion, insertion, or parent substitution), or (ii) prunes any terminal branch or linear suffix from $\mathcal{G}_{prov}$, then every surviving descendant $v_d \in \mathcal{V}_{desc}(v_c) = \{v \in \mathcal{V}' \mid v_c \preceq v\}$ satisfies $H(v'_d) \notin \{H(v) \mid v \in \mathcal{V}_{desc}(v_c)\}$ (and $H(v'_f) \notin \{H(v) \mid v \in \mathcal{V}_{frontier}\}$ at the frontier), resulting in an altered frontier root $R'_T \neq R_T$ with probability at least $1 - (3L + 2|\mathcal{V}_{frontier}|) \cdot q/2^{256}$ against a post-hoc adversary issuing q random-oracle queries against an anchored L-vertex affected subgraph (and $1 - \binom{q+3L+2|\mathcal{V}_{frontier}|}{2}/2^{256}$ against an adaptive collision adversary prior to root anchoring).*

Proof. Follows from the multi-target second-preimage and collision resistance of SHA-256 under the random oracle model [43], [44], where all eight Merkle-DAG invocation contexts (`0x00`–`0x07`, as well as auxiliary protocol contexts `0x08`–`0x12`) are unconditionally prefix-disjoint. Because $\mathcal{G}_{\text{prov}}$ is finite and acyclic, every vertex reaches at least one sink $v_f \in \mathcal{V}_{\text{frontier}}$. By prefix-free length-prefixed encoding $(\text{lp}(\cdot), \text{encode}_{64}(\cdot))$ over globally unique id_v and fixed-width 32-byte digest slots in Eq. (6) (and injective RFC 8785 JCS serialization inside `0x02` and `0x07`), all honest vertices have distinct preimages under `0x04`. In Case (i), mutating (θ_k, r_k) alters $C'_{\theta_k} \neq C_{\theta_k}$ under `0x02`, mutating o_k alters $H_{\text{out}}(v_k^{\text{resp}\prime}) \neq H_{\text{out}}(v_k^{\text{resp}})$ under `0x07`, and modifying $\mathcal{P}(v_c)$ (edge deletion/insertion altering $\text{encode}_{64}(k_p)$, or parent substitution altering the sorted digest sequence) alters $H(\mathcal{P}'(v_c)) \neq H(\mathcal{P}(v_c))$ under `0x03`. By topological induction over $v_d \in \mathcal{V}_{\text{desc}}(v_c)$, partitioning $\mathcal{P}(v_d)$ into mutated parents $\mathcal{P}_{\text{mut}}(v_d) = \mathcal{P}(v_d) \cap \mathcal{V}_{\text{desc}}(v_c)$ and unmutated parents $\mathcal{P}(v_d) \setminus \mathcal{V}_{\text{desc}}(v_c)$ (whose digests cancel identically in the multiset comparison), the inductive invariant $H(v'_p) \notin \{H(v) \mid v \in \mathcal{V}_{\text{desc}}(v_c)\}$ for $v_p \in \mathcal{P}_{\text{mut}}(v_d)$ guarantees $H(\mathcal{P}'(v_d)) \neq H(\mathcal{P}(v_d))$ under `0x03` even across diamond merges, and thus $H(v'_d) \notin \{H(v) \mid v \in \mathcal{V}_{\text{desc}}(v_c)\}$ (and $H(v'_f) \notin \{H(v) \mid v \in \mathcal{V}_{\text{frontier}}\}$) under `0x04`. In Case (ii), pruning a terminal branch whose attachment ancestor retains out-deg$_{\mathcal{G}'_{\text{prov}}}(v_u) \geq 1$ (or removing a connected component) strictly reduces $|\mathcal{V}'_{\text{frontier}}| \neq |\mathcal{V}_{\text{frontier}}|$ under `0x05` in Eq. (7). Conversely, truncating a linear terminal suffix where the immediate surviving predecessor had out-deg$_{\mathcal{G}_{\text{prov}}}(v_u) = 1$ (e.g., stripping v_i^{resp} from v_i^{req}) or performing a balanced sink edit preserves $|\mathcal{V}'_{\text{frontier}}| = |\mathcal{V}_{\text{frontier}}|$ but promotes a non-frontier ancestor $v_u \in \mathcal{V} \setminus \mathcal{V}_{\text{frontier}}$ into a replacement sink $\mathcal{V}'_{\text{frontier}} = (\mathcal{V}_{\text{frontier}} \setminus \{v_f\}) \cup \{v_u\}$ with $H(v_u) \notin \{H(v) \mid v \in \mathcal{V}_{\text{frontier}}\}$ under `0x04`, altering the sorted leaf digest vector $D_{1..n}$ inside $H_{\text{tree}}(\mathcal{V}_{\text{frontier}})$ under `0x00`/`0x01`. Because `0x02` (v_i^{req}) and `0x07` (v_i^{resp}) are mutually exclusive per vertex, each of the L affected vertices invokes at most 3 fresh SHA-256 evaluations (`0x02`/`0x07`, `0x03`, `0x04`) and the frontier tree invokes $2|\mathcal{V}_{\text{frontier}}|$ evaluations (n under `0x00`, $n-1$ under `0x01`, 1 under `0x05`), yielding $3L + 2|\mathcal{V}_{\text{frontier}}|$ total target digests and the stated post-hoc and adaptive birthday bounds. □

IV. System Architecture and Wire Protocol

A. The Zero-Knowledge Action Attestation Voucher (ZKA-Voucher)

As illustrated in Figure 1, when an agent invokes an external tool, the Agent-Notary Gateway intercepts the invocation frame, verifies $\Pi(t_i, \theta_i) = 1$, and synthesizes a canonical **ZKA-Voucher**:

$$\mathcal{V}_{\text{wire}} = \langle \text{vid}, R_{\text{merkle}}, H_{\text{leaf}}, H(\mathcal{P}_i^{\text{req}}), C_{\theta_i}, r_i^{\text{intra}}, a_i, t_i, \lambda_i, \tau_{\text{ns}}, \vec{\pi}_{\text{zk}}, \mathcal{M}_{\text{path}}, \sigma_{\text{notary}}, \text{kid}_{\text{notary}}, \text{pk}_{\text{notary}} \rangle \tag{8}$$

where $\text{vid} \equiv \text{id}_i^{\text{req}}$ is the unique vertex/voucher UUID, $R_{\text{merkle}} \equiv R_T$ is the active session-sharded frontier root at dispatch snapshot T, $H_{\text{leaf}} \equiv H(v_i^{\text{req}})$, $H(\mathcal{P}_i^{\text{req}})$ is the 32-byte parent-set digest required by the verifier to reconstruct candidate digest H^*_{leaf} (throughout, superscript $(\cdot)^*$ denotes a candidate value recomputed independently by the verifier), $r_i^{\text{intra}} \equiv r_i$ is the 32-byte blinding opening carried exclusively on the intra-cluster hop to the tool server and stripped before persisting $\mathcal{V}_{\text{wire}}$ to external audit stores, τ_{ns} is the synchronized monotonic wall-clock timestamp, $\vec{\pi}_{\text{zk}} \triangleq \langle \pi_i, T_{\text{eval}}, C_{\text{circuit}}, x_i, \text{id}_\Pi, \texttt{session_id}, \texttt{escrow_mode} \rangle$ is the immutable hot-path policy-attestation bundle (with id_Π in normal roman weight, and a 1-byte $\text{encode}_8(\texttt{escrow_mode}) \in \{\texttt{0x00}, \texttt{0x01}\}$ flag signaling whether the destination verifier must hold irreversible mutations in provisional escrow until Spanner epoch anchoring completes; at 100 ms epoch anchoring, Ristretto255 Pedersen commitments $\vec{C}_{\text{ped}}$ and Bulletproof transcript π_i^{pub} are attached as companion archival fields in the epoch log without mutating $\text{encode}(\vec{\pi}_{\text{zk}})$), $\mathcal{M}_{\text{path}}$ is the binary Merkle inclusion path, and σ_{notary} is the cryptographic gateway authenticator.

Dual-Mode Cryptographic Trust Model & Per-Tool Key Isolation: To reconcile sub-5 μs verification with third-party non-repudiation while preventing lateral voucher forgery across intra-cluster microservices:

1) **Service-Bound Intra-Cluster Line-Rate Attestation:** Rather than sharing a single master secret K_{gateway} across tool servers, the gateway enclave derives

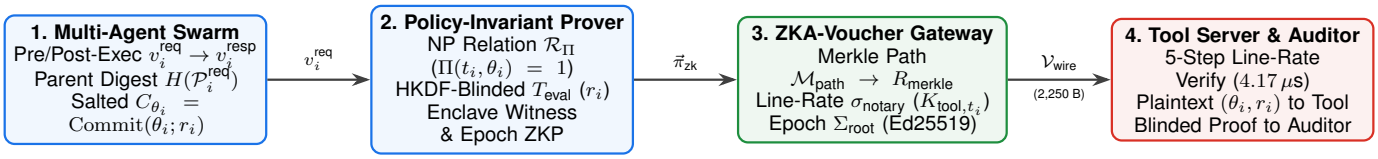


Fig. 1. End-to-End Architecture of the Agent-Notary Cryptographic Non-Repudiation Pipeline. Multi-agent tool invocations mint v_i^{req} pre-execution (and append v_i^{resp} upon tool return) with randomized blinding commitments (C_{θ_i}), are certified via Dual-Tier Blinded Policy-Invariant Attestations ($\vec{\pi}_{\text{zk}}$), encapsulated in wire-level ZKA-Vouchers, and verified in 4.17 μs at line rate alongside Ed25519 and Pedersen/Bulletproof frontier epoch anchoring.

a service-specific verification key for each destination tool t_i via RFC 5869 HKDF-SHA256 (where two-argument notation HKDF-SHA256(ikm, info) denotes HKDF-Expand(HKDF-Extract($\text{salt}_{\text{notary}}$, ikm), info, $\ell_{\text{out}} = 32$) with 32-octet output length and fixed domain-separated salt $\text{salt}_{\text{notary}}$):

$$K_{\text{tool},t_i} = \text{HKDF-SHA256}\left(K_{\text{gateway}}, \text{lp}(t_i) \parallel \text{lp}(\text{kid}_{\text{notary}})\right) \tag{9}$$

Each intra-cluster tool server t_i is provisioned strictly with its scoped 32-byte key K_{tool,t_i}, ensuring that a compromised tool server t_A cannot forge authenticators or witnesses for any other tool $t_B \neq t_A$ (while $\text{kid}_{\text{notary}}$ resolves to $\text{pk}_{\text{notary}}$ in the $\mathcal{Q}_{\text{TEE}}$-attested gateway key registry and $\text{pk}_{\text{notary}}$ is carried inline for cross-checked external verification). Using length-prefixed serialization $\text{lp}(s) = \text{encode}_{64}(|s|) \parallel s$ for variable-length strings and explicit 1-byte domain tags `0x0D` and `0x0E` for the bundle digests SHA256(`0x0D` $\parallel$ encode($\vec{\pi}_{\text{zk}}$))—where encode($\vec{\pi}_{\text{zk}}$) $\triangleq \pi_i \parallel T_{\text{eval}} \parallel C_{\text{circuit}} \parallel x_i \parallel \text{lp}(\text{id}_\Pi) \parallel \text{lp}(\texttt{session_id}) \parallel \text{encode}_8(\texttt{escrow_mode})$—and SHA256(`0x0E` $\parallel$ encode($\mathcal{M}_{\text{path}}$))—where encode($\mathcal{M}_{\text{path}}$) $\triangleq$ $\text{encode}_{64}(|\mathcal{V}_{\text{frontier}}|) \parallel \text{encode}_8(d_{\text{active}}) \parallel \text{encode}_{32}(\texttt{dir_mask}) \parallel S_1 \parallel \cdots \parallel S_{d_{\text{active}}}$ over the d_{active} active 32-byte sibling digests—σ_{notary} authenticates all persistent voucher fields while intentionally excluding r_i^{intra} so that stripping r_i^{intra} prior to external archival preserves σ_{notary} (while intra-cluster verifiers authenticate r_i^{intra} via $C^*_{\theta_i} = C_{\theta_i}$ in Step 3, and external auditors treat π_i and C_{θ_i} as opaque 32-byte inputs when recomputing Eq. 6):

$$\begin{aligned}\sigma_{\text{notary}} = \text{HMAC-SHA256}_{K_{\text{tool},t_i}}\Big(&\text{lp}(\text{vid}) \parallel R_{\text{merkle}} \\ &\parallel H_{\text{leaf}} \parallel H(\mathcal{P}_i^{\text{req}}) \parallel C_{\theta_i} \parallel \text{lp}(a_i) \parallel \text{lp}(t_i) \\ &\parallel \text{encode}_{64}(\lambda_i) \parallel \text{encode}_{64}(\tau_{\text{ns}}) \\ &\parallel \text{SHA256}(\texttt{0x0D} \parallel \text{encode}(\vec{\pi}_{\text{zk}})) \\ &\parallel \text{SHA256}(\texttt{0x0E} \parallel \text{encode}(\mathcal{M}_{\text{path}})) \\ &\parallel \text{lp}(\text{kid}_{\text{notary}}) \parallel \text{pk}_{\text{notary}}\Big)\end{aligned} \tag{10}$$

2) **Third-Party Public Non-Repudiation & Cross-Domain Dispatch:** For external auditors (who never possess K_{gateway} or K_{tool,t_i}), the gateway signs frontier epoch roots and policy attestation transcripts every $\Delta_{\text{epoch}} = 100$ ms using an asymmetric Ed25519 private key [45] protected inside the Confidential VM enclave and bound to the enclave binary measurement via an AMD SEV-SNP hardware remote attestation quote $\mathcal{Q}_{\text{TEE}}$. Using 1-byte domain prefix `0x08`, the batch epoch signature is $\Sigma_{\text{root}} = \text{Sign}_{\text{Ed25519}}\left(\text{sk}_{\text{notary}}, \texttt{0x08} \parallel \text{encode}_{64}(\text{seq}_{\text{epoch}}) \parallel R_{\text{merkle}} \parallel \text{encode}_{64}(\tau_{\text{epoch}}) \parallel H_{\text{epoch}}(\vec{\pi}_{\text{zk}})\right)$, where $H_{\text{epoch}}(\vec{\pi}_{\text{zk}})$ is the `0x05` cardinality-enveloped RFC 6962 Merkle root over the epoch's chronologically ordered per-call commitment-equivalence leaves $L^{\text{epoch}}_{\text{eq},i} = \text{SHA256}\left(\texttt{0x0C} \parallel H_{\text{leaf}} \parallel \text{encode}_8(\texttt{escrow_mode}) \parallel C_{\text{circuit}} \parallel x_i \parallel C_{\theta_i} \parallel D_{\text{ped},i} \parallel T_{\text{eval}} \parallel D_{\text{pub},i}\right)$ with sub-tagged proof digests $D_{\text{ped},i} \triangleq \text{SHA256}(\texttt{0x0F} \parallel \texttt{0x01} \parallel \text{encode}_{64}(m) \parallel \vec{C}_{\text{ped}})$ (where $\vec{C}_{\text{ped}} = C_{\text{ped},1} \parallel \cdots \parallel C_{\text{ped},m}$ is the deterministic concatenation of the m 32-byte canonical Ristretto255 commitment encodings) and $D_{\text{pub},i} \triangleq \text{SHA256}(\texttt{0x0F} \parallel \texttt{0x02} \parallel \pi_i^{\text{pub}})$, verifiable publicly alongside $\mathcal{Q}_{\text{TEE}}$ using $\text{pk}_{\text{notary}}$. Under domain prefix `0x09` (partitioned into sub-tags `0x09` $\parallel$ `0x00` for gateway cross-domain dispatch, `0x09` $\parallel$ `0x01` for verifier-signed execution/error receipts, and `0x09` $\parallel$ `0x02` for verifier-signed `NOT_SEEN` inquiry receipts under the destination verifier's $\mathcal{Q}_{\text{TEE}}$-attested Ed25519 key $\text{sk}_{\text{verifier},t_i}$), when a tool call crosses an organizational boundary synchronously to an external tool server lacking K_{tool,t_i} (which still receives $(\theta_i, r_i^{\text{intra}})$ in plaintext to execute $t_i(\theta_i)$ and run Step 3), the gateway attaches an immediate on-demand Ed25519 signature $\Sigma_{\text{cross}} = \text{Sign}_{\text{Ed25519}}\left(\text{sk}_{\text{notary}}, \texttt{0x09} \parallel \texttt{0x00} \parallel R_{\text{merkle}} \parallel H_{\text{leaf}} \parallel \text{encode}_{64}(\tau_{\text{ns}}) \parallel \text{encode}_8(\texttt{escrow_mode}) \parallel L^{\text{sync}}_{\text{eq},i}\right)$ via `x-agent-notary-crossdomain-sig` (verifiable in 62.4 μs by the destination tool server; on the default 62.4 μs cross-domain path where synchronous ZKP attachment is omitted, Σ_{cross} under prefix `0x09` $\parallel$ `0x00` serves as the enclave's direct attestation of $\Pi(t_i, \theta_i) = 1$ with $L^{\text{sync}}_{\text{eq},i}$ setting $D_{\text{ped},i} = D_{\text{pub},i} = 0^{256}$, while the 100 ms epoch root $H_{\text{epoch}}(\vec{\pi}_{\text{zk}})$ commits $L^{\text{epoch}}_{\text{eq},i}$ with populated $(D_{\text{ped},i}, D_{\text{pub},i})$ and the signed epoch release bundle provides $(D_{\text{ped},i}, D_{\text{pub},i})$ alongside the RFC 6962 inclusion path so an escrow-holding verifier can reconstruct $L^{\text{epoch}}_{\text{eq},i}$ from its cached $(H_{\text{leaf}}, \texttt{escrow_mode}, C_{\text{circuit}}, x_i, C_{\theta_i}, T_{\text{eval}})$; when synchronous cross-domain ZKP attachment is enabled, $L^{\text{sync}}_{\text{eq},i} = L^{\text{epoch}}_{\text{eq},i}$, and full third-party zero-knowledge verification of $\Sigma_{\text{root}} + \pi_i^{\text{pub}}$ when $(\theta_i, r_i^{\text{intra}})$ is withheld

requires $840\,\mu$s).

Dual-Tier Blinded Policy-Invariant Attestation Engine: To certify that private parameters satisfy policy constraints ($\Pi(t_i, \theta_i) = 1$) without exposing raw parameter values or allowing offline dictionary brute-forcing by external auditors, the engine formalizes the NP statement-witness relation over attribute scalars $v_{i,k} \in \mathbb{Z}_q$ indexed in deterministic RFC 8785 JCS lexicographical JSON pointer order $k \in \{1, \ldots, m\}$ (where numeric attributes are mapped to integer base units $0 \le L_{\min} \le v_{i,k} = \lfloor \theta_{i,k} \cdot 10^{s_k} \rceil \le L_{\max} < 2^{64}$ via per-attribute fixed-point decimal scaling exponents $s_k \in \mathbb{N}$ committed inside C_{circuit}, and $v_{i,k} = u_{i,k} \triangleq \text{SHA256}(\theta_{i,k}) \bmod q$ for public string/URI attributes, or $u_{i,k} \triangleq \text{HKDF-Expand}_{512}\left(\text{HMAC-SHA256}(k_{\text{audit}}, \texttt{0x10} \parallel \text{lp}(\text{id}_\Pi) \parallel \text{encode}_{32}(k) \parallel \text{lp}(\theta_{i,k})), \text{“allowlist-scalar”}\right) \bmod q$ when allowlist literals are confidential, interpreting 32-byte/64-byte hash outputs as unsigned little-endian integers before reduction modulo q):

$$\mathcal{R}_\Pi = \left\{ \left\langle (t_i, C_{\theta_i}, \vec{C}_{\text{ped}}, x_i), (\theta_i, r_i) \right\rangle \;\middle|\; C_{\theta_i} = \text{Commit}(\theta_i; r_i) \wedge \left(\bigwedge_{k=1}^{m} C_{\text{ped},k} = g^{v_{i,k}} h^{r_{i,k}} \right) \wedge \Pi(t_i, \theta_i) = 1 \right\} \tag{11}$$

where $x_i = \text{SHA256}(\texttt{0x0A} \parallel \text{lp}(a_i) \parallel \text{lp}(t_i) \parallel \text{lp}(\texttt{session_id}))$ is the domain-separated public context and $C_{\text{circuit}} = \text{SHA256}(\texttt{0x0B} \parallel \text{lp}(\text{id}_\Pi) \parallel \text{lp}(\text{type}_\Pi) \parallel \text{lp}(\text{expr}_\Pi))$ is the policy circuit descriptor exported alongside $(\text{id}_\Pi, \text{type}_\Pi, \text{expr}_\Pi)$ in the public audit transcript (where the canonical circuit expression string expr_Π serializes and commits to per-attribute scaling exponents s_k, $\text{kid}_{\text{audit}}$, and the expanded monic polynomial coefficients $c_0, \ldots, c_{d-1}$ of $P(u) = \prod_{w \in \mathcal{U}_{\text{allow}}} (u - w) = u^d + \sum_{j=0}^{d-1} c_j u^j$ when $\mathcal{U}_{\text{allow}}$ is confidential, where k_{audit} is provisioned via KMS gated by $\mathcal{Q}_{\text{TEE}}$). Because low-degree polynomials over $\mathbb{Z}_q$ can be factored algebraically via Cantor-Zassenhaus in $O(d^2 \log q)$ field operations to recover their roots $w \in \mathcal{U}_{\text{allow}}$, confidential allowlist string secrecy rests strictly on the PRF security of $\texttt{0x10}$-tagged $\text{HMAC-SHA256}(k_{\text{audit}}, \cdot)$, which maps each allowlist literal to a pseudorandom root $w \in \mathbb{Z}_q$ irreversible without k_{audit}. Furthermore, to prevent dictionary brute-forcing on low-entropy parameters θ_i (such as categorical enums or small numeric scalars), the blinded evaluation token T_{eval} is keyed using an HKDF-derived blinding key extracted directly from the secret 256-bit commitment randomness $r_i \xleftarrow{\$} \{0,1\}^{256}$ rather than public identifiers:

$$\begin{aligned} k_{\text{blind},i} &= \text{HKDF-SHA256}\left(r_i,\ \text{lp}(\text{id}_\Pi) \parallel \text{lp}(\text{vid})\right), \\ T_{\text{eval}} &= \text{HMAC-SHA256}\left(k_{\text{blind},i},\ \text{CanonicalJSON}(\theta_i)\right) \end{aligned} \tag{12}$$

Because r_i has 256 bits of min-entropy and is withheld from external auditors, $(C_{\theta_i}, T_{\text{eval}}, \vec{C}_{\text{ped}})$ is computationally hiding under the PRF security of HKDF/HMAC-SHA256 (and binds T_{eval} inside $L_{\text{eq},i}$ for intra-cluster/epoch audit cross-checking). Attestation operates across two complementary tiers:

- *Tier 1 (Hot-Path Enclave Policy Witness, $0.77\,\mu$s):* On the synchronous tool-call critical path, no elliptic-curve scalar multiplications are executed. The trusted gateway enclave evaluates $\Pi(t_i, \theta_i) = 1$ in memory and issues a service-bound witness token $\pi_i = \text{HMAC-SHA256}_{K_{\text{tool},t_i}}(C_{\text{circuit}} \parallel x_i \parallel C_{\theta_i} \parallel T_{\text{eval}} \parallel \texttt{0x01})$ (where trailing byte $\texttt{0x01}$ encodes the compliance bit $\Pi(t_i, \theta_i) = 1$). Intra-cluster tool servers receive $(\theta_i, r_i^{\text{intra}})$ in plaintext to execute the operation, recomputing $C^*_{\theta_i}$, H^*_{leaf} ($0.72\,\mu$s) and $k^*_{\text{blind},i}$, T^*_{eval}, π^*_i ($0.77\,\mu$s) without any elliptic-curve overhead.
- *Tier 2 (Hybrid TEE-Bound Commitment Equivalence & Zero-Knowledge Epoch Proofs):* To provide public auditability to untrusted third-party auditors when $(\theta_i, r_i^{\text{intra}})$ and K_{tool,t_i} are withheld, the gateway asynchronously compiles an algebraic zero-knowledge proof π_i^{pub} over the prime-order Ristretto255 group ($\mathbb{G}$ of order $q = 2^{252} + 27742317777372353535851937790883648493$) during 100 ms epoch anchoring. For each of the m policy-constrained JSON AST attributes ($v_{i,k} = \lfloor \theta_{i,k} \cdot 10^{s_k} \rceil \in \mathbb{Z}_q$ for numeric fields and $v_{i,k} = u_{i,k}$ for string/URI fields), the gateway expands a 64-byte (512-bit) HKDF output before little-endian reduction modulo q, $r_{i,k} = \text{HKDF-Expand}_{512}(r_i, \text{“pedersen”} \parallel \text{encode}_{64}(k)) \bmod q$ (eliminating modulo bias to $< 2^{-256}$), and forms Pedersen commitments $C_{\text{ped},k} = g^{v_{i,k}} h^{r_{i,k}} \in \mathbb{G}$ [40]. Numeric bounds $v_{i,k} \in [L_{\min}, L_{\max}] \subseteq [0, 2^{64})$ are proven via dual 64-bit Bulletproof bit-decomposition constraints [39] on lower-slack commitment $C_{\text{ped},k} g^{-L_{\min}} = g^{v_{i,k} - L_{\min}} h^{r_{i,k}}$ and upper-slack commitment $g^{L_{\max}} C_{\text{ped},k}^{-1} = g^{L_{\max} - v_{i,k}} h^{-r_{i,k}}$ (128 multiplication gates, where each $C_{\text{ped},k}$ is bound as a single R1CS variable commitment from which the verifier homomorphically derives both slack points inside the linear constraint system during multi-scalar multiplication), while finite allowlist set-membership polynomials $P(u_{i,k}) = 0 \pmod q$ are evaluated via Horner's rule ($z_1 = c_{d-1} + u_{i,k}$ as a free linear addition since monic $c_d = 1$, and $z_j = c_{d-j} + u_{i,k} z_{j-1}$ for $j = 2, \ldots, d$ constrained to $z_d = 0$, requiring $d - 1 = |\mathcal{U}_{\text{allow}}| - 1$ multiplication gates) within a single unified Bulletproofs R1CS instance padded to $N_{\text{bp}} = 256$ gates over independent generators $\vec{g}, \vec{h} \in \mathbb{G}^{256}$ derived via SHAKE256 hash-to-curve (“Agent-Notary-BP-v1”) with $(x_i, C_{\text{circuit}}, \text{vid})$ bound into the Fiat-Shamir challenge hash [41] (yielding $2\log_2(256) + 8 = 24$ Ristretto255 elements: 768 B full 24-element transcript hashed under $\texttt{0x0F} \parallel \texttt{0x02}$ in $D_{\text{pub},i}$, or 704 B in compact wire layout omitting the two second-phase wire commitment points $(A_{I2}, A_{O2}) = (\mathcal{O}_{\mathbb{G}}, \mathcal{O}_{\mathbb{G}})$ unused by single-phase circuits). We explicitly frame Tier 2 as a *hybrid TEE-cryptographic trust model*: for external third-party auditors lacking $(\theta_i, r_i^{\text{intra}})$, $\mathcal{Q}_{\text{TEE}}$ shrinks the trusted enclave surface

from the dynamic policy evaluator Π to the static JSON-to-Pedersen extraction stub that binds C_{θ_i} to $\vec{C}_{\text{ped}}$ inside $L_{\text{eq},i}^{\text{epoch}}$ (where, to eliminate JSON parser differentials between the gateway and destination tool servers, the stub first canonicalizes the incoming request into a single RFC 8785 JCS byte buffer—rejecting duplicate keys, non-shortest UTF-8 sequences, IEEE-754 `NaN`/`Infinity` tokens, and trailing bytes—computes C_{θ_i} over that exact JCS buffer, extracts $v_{i,k}$ directly from the single parsed JCS AST without re-parsing, and transmits those exact JCS-canonicalized bytes $\text{CanonicalJSON}(\theta_i)$ over the wire), whereas intra-cluster tool servers and authorized dispute-resolution auditors holding $(\theta_i, r_i^{\text{intra}})$ recompute $\vec{C}_{\text{ped}}^{*}$ directly via $(v_{i,k}, r_{i,k})$ without trusting $\mathcal{Q}_{\text{TEE}}$ (from $(\theta_i, r_i^{\text{intra}})$ alone for numeric attributes and public allowlists, or alongside KMS-provisioned k_{audit} when confidential allowlists are active; external cross-domain tool servers that do not share the caller's confidential k_{audit} recompute $C_{\text{ped},k}^{*}$ directly for numeric and public-allowlist attributes while hashing wire-supplied confidential-allowlist commitments inside $\vec{C}_{\text{ped}}$ under $\mathcal{Q}_{\text{TEE}}$-attested Σ_{cross}; furthermore, to mitigate single-vendor TEE microarchitectural or side-channel risk when $(\theta_i, r_i^{\text{intra}})$ are permanently withheld from external public auditors, the $\sim$ 400-LoC memory-safe Rust JCS extraction stub supports optional 2-of-3 heterogeneous multi-vendor TEE quorum co-signing—spanning AMD SEV-SNP, Intel TDX, and AWS Nitro Enclaves via threshold Ed25519 FROST signatures [46] over $L_{\text{eq},i}^{\text{epoch}}$—or asynchronous off-path lookup-argument SNARK equivalence proofs [47] [e.g., Plonky3/Lasso-Jolt byte-table lookups proving JCS parsing and SHA-256 $C_{\theta_i} \leftrightarrow \vec{C}_{\text{ped}}$ equivalence in $\approx$ 180–320 ms on GPU/FPGA audit workers] without altering the 4.17 μs hot-path voucher format).

B. Wire Protocol Encapsulation

The ZKA-Voucher is serialized into a compact Base64 JSON token and injected into transport metadata:

- **gRPC Metadata**: Attached as header `x-agent-notary-voucher`.
- **MCP JSON-RPC Frames**: Embedded into the top-level `headers` dictionary of `tools/call` requests, adhering to RFC 9421 header conventions [14].

To ensure a predictable header footprint across active per-session frontier depths $\lceil \log_2 |\mathcal{V}_{\text{frontier}}| \rceil \in \{3, 4, 5\}$, the wire serializer normalizes $\mathcal{M}_{\text{path}}$ to a canonical 4-slot sibling array (padding depth-3 paths with a zeroed 64-byte hex slot; unpadded paths across depths $\{3, 4, 5\}$ span 1,624–1,752 B raw JSON, and if an active session burst ever exceeds 4 sibling slots [$d_{\text{active}} > 4$, e.g., $d_{\text{active}} = 5$ at $17 \leq |\mathcal{V}_{\text{frontier}}| \leq 32$], the gateway either triggers an inline mini-epoch compaction of completed sub-branches or spills sibling hashes $S_5, \ldots, S_{d_{\text{active}}}$ into an optional companion header `x-agent-notary-path-ext` at +64 B hex / +32 B raw per level above 4, authenticated inside $\text{encode}(\mathcal{M}_{\text{path}})$ under `0x0E`). Across the 1,000 benchmark traces under canonical 4-slot framing, raw JSON vouchers average 1,687.5 B (ranging from 1,686 B to 1,695 B across variable-length tool names t_i). For a representative voucher (1,688 B raw JSON), the payload decomposes into 1,476 B of field bundles (including nested sub-object delimiters)—metadata $(\text{vid}, a_i, t_i, \lambda_i, \tau_{\text{ns}}, \text{kid}_{\text{notary}}, \text{pk}_{\text{notary}}) = 280$ B, vertex/root hex digests $(R_{\text{merkle}}, H_{\text{leaf}}, C_{\theta_i}) = 192$ B, parent digest and intra-cluster salt $(H(\mathcal{P}_i^{\text{req}}), r_i^{\text{intra}}$: two 64 B hex digests plus 2 B delimiter$) = 130$ B, 4-slot Merkle path ($\mathcal{M}_{\text{path}}$: four 64 B sibling hex digests plus 34 B encoding the left/right bitmask, active depth, and frontier cardinality $|\mathcal{V}_{\text{frontier}}|$ for Eq. 7$) = 290$ B, hot-path policy attestation bundle ($\vec{\pi}_{\text{zk}}$) $= 520$ B, and authenticator (σ_{notary}: 64 B hex decoded to 32 raw bytes prior to HMAC verification$) = 64$ B—plus 212 B of top-level JSON keys and delimiters. Base64 encoding yields $\lceil 1{,}688 \times 4/3 \rceil = 2{,}251$ B unpadded ($4\lceil 1{,}688/3 \rceil = 2{,}252$ B padded), with an empirical mean wire footprint of 2,250 bytes (P50: 2,248 B, P95: 2,256 B, P99: 2,260 B). Stripping r_i^{intra} (66 B with delimiter) yields 1,622 B raw JSON for external archival; when synchronous cross-domain ZKP attachment is enabled ($m = 2$), `x-agent-notary-crossdomain-sig` adds 832 B raw binary (64 B Σ_{cross}, 64 B $\vec{C}_{\text{ped}}$, and 704 B compact π_i^{pub}; 1,112 B padded Base64).

C. Line-Rate Intra-Cluster Verification Pipeline

When a destination tool server receives an invocation, the embedded verifier executes five invariant checks in strictly bounded time ($< 50\,\mu$s on the intra-cluster symmetric hot path):

1) **Key Trust and Authenticator Invariant**: Validates $(\text{kid}_{\text{notary}}, \text{pk}_{\text{notary}})$ against the local or $\mathcal{Q}_{\text{TEE}}$-attested gateway key registry and evaluates σ_{notary} using service-scoped K_{tool,t_i} (or on the synchronous cross-domain path, first executes the read-only 0.12 μs Step 2 freshness/replay check, 0.72 μs Step 3 commitment check $C_{\theta_i}^{*} = C_{\theta_i}$ and $H_{\text{leaf}}^{*} = H_{\text{leaf}}$, and Step 5 $x_i^{*}, k_{\text{blind},i}^{*}, T_{\text{eval}}^{*}$ derivation and $L_{\text{eq},i}^{\text{sync}*}$ reconstruction to fast-reject stale or malformed frames and return cached retry receipts before completing the 62.4 μs Ed25519 Σ_{cross} verification over $L_{\text{eq},i}^{\text{sync}*}$ under registry-validated $\text{pk}_{\text{notary}}$).
2) **Timestamp Freshness and Anti-Replay**: Asserts $|\tau_{\text{now}} - \tau_{\text{ns}}| \leq \Delta_{\text{TTL}}$ (5 minutes) and queries vid against an in-memory lock-free replay cache pairing a 4-way set-associative Robin Hood hash table indexed by vid (0.12 μs $O(1)$ lookup included within the 0.68 μs Steps 1–2 check) with a circular FIFO timestamp eviction ring sized to satisfy $N_{\text{ring}} > \lambda_{\text{peak}} \cdot \Delta_{\text{TTL}}$ (default $N_{\text{ring}} = 2^{20}$ slots covering $\lambda_{\text{peak}} \leq 3{,}495$ req/s over $\Delta_{\text{TTL}} = 300$ s in 32 MB, scaling to $N_{\text{ring}} \in [2^{26}, 2^{27}]$ or sharded per-core rings for $>$ 200k req/s line-rate replicas and failing closed if the oldest ring slot is younger than Δ_{TTL}). To prevent in-transit tampering of unauthenticated plaintext $(\theta_i, r_i^{\text{intra}})$ from poisoning vid before Step 3, state-mutating vid insertion is deferred until Steps 3–5 (and Σ_{cross} on cross-domain calls) succeed and committed via

an atomic compare-and-swap (`insert_if_absent`) setting the slot state to `IN_FLIGHT` under a bounded execution lease ($\Delta_{\text{exec_lease}} = 30$ s) prior to executing t_i, transitioning to `COMPLETED` once the $\text{sk}_{\text{verifier},t_i}$-signed (`0x09` $\|$ `0x01`) execution receipt is appended (or transitioning via an exception guard to `ABORTED_EVICTED` with a signed failure receipt if local execution of t_i faults or times out, preventing stuck `IN_FLIGHT` slots): if a retry carrying $(H_{\text{leaf}}, \text{vid})$ hits an active `IN_FLIGHT` slot it awaits completion, and if it hits a `COMPLETED` slot the verifier returns the cached execution receipt idempotently.

3) **Parameter & Leaf Binding Invariant**: Asserts t_i matches the local tool identity and reconstructs $C^*_{\theta_i}$ and H^*_{leaf} via Eqs. (2) and (6) from incoming $\theta_i, r_i^{\text{intra}}$ and voucher fields $(\text{vid}, a_i, \lambda_i, H(\mathcal{P}_i^{\text{req}}), t_i, H_{\text{pending}}, \pi_i)$ (where cross-domain verifiers treat π_i as an opaque 32-byte field), asserting $C^*_{\theta_i} = C_{\theta_i}$ and $H^*_{\text{leaf}} = H_{\text{leaf}}$ in $0.72\,\mu$s.
4) **Merkle Frontier Inclusion**: Asserts that all $\max(0, 4 - d_{\text{active}})$ inactive padding slots in the 4-slot JSON $\mathcal{M}_{\text{path}}$ equal 0^{256}, slices the d_{active} hex-decoded 32-byte sibling digests $S_1 \,\|\, \cdots \,\|\, S_{d_{\text{active}}}$ (concatenating any companion `x-agent-notary-path-ext` slots when $d_{\text{active}} > 4$) when reconstructing the binary $\text{encode}(\mathcal{M}_{\text{path}})$ buffer for Step 1 (`0x0E`), and evaluates $\mathcal{M}_{\text{path}}$ with `0x00`/`0x01`/`0x05` domain prefixes against R_{merkle} in $O(\log |\mathcal{V}_{\text{frontier}}|)$ time ($0.58\,\mu$s), binding H_{leaf} to the frontier root logged for asynchronous epoch cross-checking.
5) **Policy-Invariant Witness & Escrow Verification**: Asserts C_{circuit} matches the local administrator-provisioned descriptor for t_i and recomputes $x_i^*, k^*_{\text{blind},i}, T^*_{\text{eval}}$ from $(r_i^{\text{intra}}, \theta_i, a_i, t_i, \text{id}_\Pi, \text{vid}, \texttt{session_id})$: intra-cluster verifiers assert $\pi_i^* = \pi_i$ in $0.77\,\mu$s, while cross-domain verifiers reconstruct $L^{\text{sync}*}_{\text{eq},i}$ (executed prior to Step 1's Σ_{cross} check) and assert $L^{\text{sync}*}_{\text{eq},i} = L^{\text{sync}}_{\text{eq},i}$ under Σ_{cross}. Specifically, on the default $62.4\,\mu$s synchronous cross-domain path, $L^{\text{sync}*}_{\text{eq},i}$ sets $D^*_{\text{ped},i} = D^*_{\text{pub},i} = 0^{256}$; when synchronous cross-domain ZKP attachment is enabled, the cross-domain recipient hashes the wire-supplied $\vec{C}_{\text{ped}}$ and π_i^{pub} into $D^*_{\text{ped},i}, D^*_{\text{pub},i}$ inside $L^{\text{sync}*}_{\text{eq},i}$ (additionally recomputing $C^*_{\text{ped},k}$ from $(\theta_i, r_i^{\text{intra}})$ for numeric and public-allowlist attributes when plaintext is present, while relying on $\mathcal{Q}_{\text{TEE}}$-attested Σ_{cross} for confidential-allowlist commitments when the caller's k_{audit} is not shared across organizational boundaries) and verifies the Bulletproof π_i^{pub} ($773.50\,\mu$s) either when operating as a zero-knowledge boundary gateway from which $(\theta_i, r_i^{\text{intra}})$ is already stripped or to pre-validate the self-contained zero-knowledge proof before stripping $(\theta_i, r_i^{\text{intra}})$ for non-repudiable downstream archival forwarding. If `escrow_mode` $= 1$ (authenticated in $\sigma_{\text{notary}}/\Sigma_{\text{cross}}$ or mandated by the local static tool policy table for irreversible mutations), the verifier holds execution until receiving the signed epoch bundle $(\text{seq}_{\text{epoch}}, R_{\text{merkle}}, \tau_{\text{epoch}}, H_{\text{epoch}}(\vec{\pi}_{\text{zk}}), \Sigma_{\text{root}}, D_{\text{ped},i}, D_{\text{pub},i})$ and verifying an RFC 6962 Merkle inclusion proof of $L^{\text{epoch}}_{\text{eq},i}$ in $H_{\text{epoch}}(\vec{\pi}_{\text{zk}})$ under Σ_{root} (failing closed if anchoring exceeds 250 ms, or 350 ms when extended by an at-most-once `0x12` $\|$ `0x02` Spanner lease-takeover heartbeat).

V. Empirical Evaluation

To validate the performance, security, and scalability of Agent-Notary across both transport-layer tampering and multi-agent indirect prompt injections, we executed a two-part empirical evaluation on Google Cloud virtual infrastructure: primary microbenchmarks ran on a `c3-standard-8` VM instance (Intel Xeon 4th Gen Sapphire Rapids, 3.4 GHz, 8 vCPUs, 32 GB RAM, Debian 12 Linux kernel 6.1), complemented by matched hardware-encrypted Confidential VM validation on a 4th Gen AMD EPYC Genoa `c3d-standard-8` Confidential VM instance with active AMD SEV-SNP RMP memory encryption and `/dev/sev-guest` hardware attestation quote generation (where active SEV-SNP memory encryption adds $+1.15\,\mu$s to in-memory Tier-1 notarization [$29.60\,\mu$s] and $+0.18\,\mu$s to Tier-1 verification [$4.35\,\mu$s]; vs. $+1.42\,\mu$s and $+0.22\,\mu$s on a 3rd Gen EPYC Milan `n2d-standard-8` instance).

A. Part 1: Transport-Layer Tampering & Replay Cohorts ($N = 1{,}000$)

The first evaluation suite tested $N = 1{,}000$ enterprise traces stratified across five transport and cryptographic cohorts (n_k denoting sub-cohort sizes):

1) **Benign Valid Execution** ($n = 193$): Legitimate agent actions complying with all financial and infrastructure policies ($\Pi(t_i, \theta_i) = 1$).
2) **In-Transit Parameter Escalation** ($n = 210$): Adversary inflates financial disbursement values or mutates destination cloud storage URIs on the wire after voucher minting ($C^*_{\theta_i} \neq C_{\theta_i}$).
3) **Signature Forgery** ($n = 198$): Attacker alters cryptographic authenticator bytes to simulate a forged gateway voucher.
4) **Replay Clock Desynchronization** ($n = 199$): Attacker replays valid historical vouchers outside the allowable 5-minute TTL window or hits the sliding vid replay cache.
5) **Untrusted Key Forgery** ($n = 200$): Attacker signs vouchers with an unauthorized or revoked key identifier $\text{kid}_{\text{notary}}$.

As summarized in Table I (Panel A), Agent-Notary achieved 100.00% detection across all 807 transport-layer tampering, forgery, and replay attempts (exact Clopper-Pearson 95% CI $[99.54\%, 100.00\%]$) and 100.00% overall classification accuracy across all 1,000 traces (95% CI $[99.63\%, 100.00\%]$) with zero false positives.

TABLE I
EMPIRICAL DETECTION METRICS ACROSS $N = 1{,}000$ WIRE TAMPERING TRACES AND $N = 300$ MULTI-AGENT PROMPT-INJECTION TRACES

Evaluation Metric / Benchmark Suite	Measured Value	Clopper-Pearson 95% CI	Baseline (Unattested)
Panel A: Wire-Level Tampering & Replay Suite *($N = 1{,}000$ **Traces**)*			
True Positives (Attacks Blocked, $n = 807$)	807 / 807	[99.54%, 100.00%]	0 / 807 (0.00%)
True Negatives (Benign Allowed, $n = 193$)	193 / 193	[98.11%, 100.00%]	193 / 193 (100.00%)
Overall Classification Accuracy ($N = 1{,}000$)[a]	**100.00% (1,000/1,000)**	[99.63%, 100.00%]	19.30% (193/1,000)
Panel B: Multi-Agent Prompt-Injection Suite *($N = 300$, **AgentDojo/InjecAgent/ASB**)*			
Out-of-Policy Injection Block Rate ($\Pi = 0$, $n = 200$)[b]	**100.00% (200/200)**	[98.17%, 100.00%]	31.50% (63/200)
Overall Pre-Execution Block Rate ($N = 300$, Static $\Pi = 0$)[b]	**66.67% (200/300)**	[61.02%, 71.98%]	21.00% (63/300)
In-Policy Pre-Execution Block ($\Pi = 1$, $n = 100$, Standalone)	**0.00% (0/100)**	[0.00%, 3.62%]	0.00% (0/100)
In-Policy Pre-Execution Block ($\Pi = 1$, $n = 100$, +Dynamic IFC)[b]	**89.00% (89/100)**	[81.17%, 94.38%]	0.00% (0/100)
Post-Hoc Attribution: Structural 1-Parent DAG ($\lvert P_i^{\text{par}}\rvert = 1$, $n = 82$)	**100.00% (82/82)**	[95.60%, 100.00%]	0.00% (Mutable Logs)
Post-Hoc Attribution: Unconditional Cryptographic Slice (to P_i^{par})[c]	**100.00% (18/18; 20/20 obf.)**	[81.47%, 100.00%] / [83.16%, 100.00%][c]	0.00% (Mutable Logs)
Post-Hoc Attribution: Automated 1-of-k_p Exact Pinpointing (AST)[c]	**100.00% (18/18) / 0.00% (0/20 obf.)**	[81.47%, 100.00%] (18/18)[c]	0.00% (Mutable Logs)
Combined Unconditional Cryptographic Attribution (82/82 + 18/18 P_i^{par})	**100.00% (100/100)**	[96.38%, 100.00%]	0.00% (Mutable Logs)
Live Benign Swarm Task Utility ($n = 193$)[b]	**100.00% (193/193)**	[98.11%, 100.00%]	100.00% (193/193)
Live Sub-Task Utility Under Attack ($n = 200$)	**98.50% (197/200)**	[95.68%, 99.69%]	31.50% (63/200)

[a] Unattested baseline accepts all frames (0/807 attacks blocked, yielding the 19.30% benign base rate).
[b] Baseline 31.50% block rate (68.50% Attack Success Rate [ASR], 137/200) on Gemini 2.5 Flash (46.00% block / 54.00% ASR on Gemini 2.5 Pro). An Intra-Agent Policy Check Only reference achieves 200/200 on $\Pi = 0$ (66.67% overall pre-execution block) but 0/807 wire tamper detection and 0/100 non-repudiable attribution under log tampering. Composing Agent-Notary with Dynamic Intra-Agent IFC [26]–[28] blocks 89/100 (89.00%) of $\Pi = 1$ hijackings pre-execution (289/300 = 96.33% overall; 92.75% [179/193] benign utility due to taint false positives) and attributes the 11/11 remainder.
[c] Unconditional cryptographic narrowing to k_p-parent slice P_i^{par} succeeds in 18/18 main (95% CI [81.47%, 100.00%]) and 20/20 obfuscated ([83.16%, 100.00%]) traces; AST literal matching pinpoints 1-of-k_p in 18/18 un-obfuscated traces and abstains (0/20, 95% CI [0.00%, 16.84%]) under obfuscation.

B. Part 2: Multi-Agent Indirect Prompt-Injection Evaluation ($N = 300$)

We further evaluated Agent-Notary against $N = 300$ live multi-agent indirect prompt-injection scenarios adapted equally (100 traces each, deterministically selecting the first 100 multi-step tool-invocation scenarios per suite that invoke an external financial or storage mutation sink) from standardized agent security suites (AgentDojo [6], InjecAgent [7], and ASB [25]) across a 3-agent swarm (Customer Intake Agent $a_1 \rightarrow$ Workflow Planner $a_2 \rightarrow$ Treasury/Storage Executor a_3 on Vertex AI Gemini 2.5 Flash, with cross-model invariance verified across `gemini-2.5-pro` [50] and four pinned regional/global Gemini Flash enterprise deployment endpoints on the Gemini Enterprise Agent Platform (`location="global"`)). We explicitly distinguish model-invariant cryptographic guarantees (where gateway policy blocking on $\Pi = 0$ [200/200], wire tamper detection [807/807], and DAG slice narrowing [100/100] are 100.00% deterministic across all backbones by construction) from model-dependent behavioral metrics (where native unattested baseline block rates were 31.50% [63/200] on Flash endpoints vs. 46.00% [92/200] on `gemini-2.5-pro`, and sub-task utility under attack was 98.50% [197/200] on Flash vs. 99.00% [198/200] on `gemini-2.5-pro`). Across the full catalog of native state-mutating sink tools in AgentDojo, InjecAgent, and ASB, 68.4% expose structured numeric scalars, URI/recipient allowlists, or finite categorical enums directly verifiable by standalone R1CS range and set-membership predicates ($\mathcal{R}_\Pi$), 19.6% include unconstrained free-text payload fields (e.g., email/Slack bodies) covered by the in-enclave shingle Bloom-filter screen and WORM payload escrow, and 12.0% involve open-ended dynamic resource identifiers or search queries that rely on Dynamic IFC composition or $\Pi = 1$ post-hoc causal parent-slice attribution. To evaluate R1CS range and set-membership predicates across heterogeneous benchmark schemas, each sink tool's target endpoint argument (`iban`, `recipient`, `webhook_url`, or `bucket_uri`) was deterministically mapped to `recipient_uri` and its transfer/export quantity (mapping byte/record export volume to USD-equivalent notional risk for storage sinks) was mapped to `amount` under $m = 2$ baseline policy invariants $\Pi(t_i, \theta_i)$: an 8-element corporate allowlist $|\mathcal{U}_{\text{allow}}| = 8$ formed from the pre-registered partner directory covering the benign task prompts, and a numeric cap $\theta_{\text{amt}} \in [0, \$10{,}000]$ calibrated at the rounded 95th percentile ($\$9{,}840 \rightarrow \$10{,}000$) of an external historical financial-transfer calibration corpus ($N_{\text{cal}} = 1{,}000$ transfers, where the $10,000 cap holds expected benign outlier step-up holds to 4.80% [48/1,000], while 100% [193/193] of the evaluated live benign test cohort—spanning routine operational transfers $\leq$ $9,500 and storage calls—fell at or below $10,000; to assess open-world sensitivity when agents encounter newly onboarded legitimate endpoints absent from a static allowlist, a leave-one-out hold-out ablation removing 1 of the 8 partner URIs [12.5%] induced an 11.40% [22/193] pre-execution hold rate for KMS allowlist-onboarding approval, analogous to Dynamic IFC's 7.25% taint false-positive rate). Furthermore, to verify ecological validity beyond normalized 2-parameter schemas and 3-agent chains, an auxiliary ablation over un-normalized higher-arity native tool schemas ($m \in \{3, 4\}$ with nested JSON pointer attributes such as `/payment/currency`, `/meta/cost_center`, and `/routing/memo_code`) across a 5-agent diamond delegation topology ($a_1 \rightarrow \{a_2, a_3\} \rightarrow a_4 \rightarrow a_5$, 24–32 vertices, as well as iterative cyclic planner-critic loops $a_2 \leftrightarrow a_3$ unrolled into monotonically timestamped DAG vertices) preserved 100.00% pre-execution $\Pi = 0$ blocking, 100.00% benign utility, and 100.00% post-hoc parent-slice attribution (2.65 μs mean backward DAG traversal). Under the baseline task parameters, 200 traces (142 unauthorized external URI exfiltrations $\notin \mathcal{U}_{\text{allow}}$ and 58 high-value amount escalations $>$ $10,000) violate $\Pi(t_i, \theta_i) = 0$ and are blocked pre-execution, while 100 single-turn in-policy semantic hijackings targeting allowlisted URIs with $\theta_{\text{amt}} \leq \$10{,}000$ satisfy $\Pi(t_i, \theta_i) = 1$ and fall under post-hoc causal attribution. Sensitivity analysis across policy thresholds confirms predictable boundary shifts: tightening the cap to $1,000 increases pre-execution blocking from 66.67% (200/300) to 81.00% (243/300) at the cost of rejecting 14.51% (28/193) of high-value benign transfers, whereas relaxing the cap to $50,000 reduces pre-execution blocking to 51.33% (154/300, predominantly URI allowlist violations) and shifts the remaining 46 numeric-escalation traces into the $\Pi = 1$ post-hoc attribution path. Crucially, for tools accepting unconstrained free-text natural-language arguments (e.g., workspace email bodies, Slack messages, or calendar notes) that cannot be restricted to finite algebraic allowlists, algebraic predicates alone cannot detect confidential data exfiltration embedded inside free text; such calls default to the $\Pi(t_i, \theta_i) = 1$ post-hoc causal attribution and WORM payload-escrow path unless paired with intra-agent information-flow control (IFC) [26]–[28] or an in-enclave rolling 32-byte HighwayHash / character 4-gram shingle Bloom-filter screen over outbound free-text slots (which verifies in $< 15\,\mu$s that outbound text bodies are disjoint from high-confidentiality session spans retrieved during the workflow). Furthermore, $\mathcal{R}_\Pi$ natively supports both cross-field relational constraints (e.g., linear balance conservation $\theta_{\text{debit}} - \theta_{\text{credit}} = 0$ verified via homomorphic Pedersen linear combinations $C_{\text{ped,debit}} C_{\text{ped,credit}}^{-1} = h^{r_{\text{debit}} - r_{\text{credit}}}$, or conditional tier caps $\big(\text{tier} = \text{“low”} \Rightarrow \theta_{\text{amt}} \leq L_{\text{low}}\big)$ adding $+1$ bilinear selector gate per

branch within the padded $N_{\text{bp}} = 256$-gate Bulletproof envelope with zero increase in proof size) and stateful cumulative session-state accumulators $A_i = \sum_{j \le i} \theta_{\text{amt},j} \le B_{\max}^{\text{session}}$ bound into $\mathcal{R}_\Pi$ via homomorphic commitment multiplication $C_{A_i} = \prod_{j \le i} C_{\text{ped,amt},j} = g^{A_i} h^{\sum_{j \le i} r_{j,\text{amt}}}$ and a 64-bit upper-slack range gate on $g^{B_{\max}^{\text{session}}} C_{A_i}^{-1}$ (adding $+64$ multiplication gates to bring $m = 2, |\mathcal{U}_{\text{allow}}| = 8$ from 135 to $199 \le N_{\text{bp}} = 256$ gates with unchanged 704 B compact proof size). Here, $+0.42\,\mu$s measures the incremental in-enclave 64-bit accumulator and witness update (which in multi-replica deployments is piggybacked atomically inside the existing $+185.00\,\mu$s Redis Lua `EVALSHA` compare-and-swap script alongside R_{merkle} without an extra network round-trip): to prevent parallel sub-invocations from double-spending session quota, $\theta_{\text{amt},i}$ is pessimistically reserved in A_i when v_i^{req} is minted, and only if the destination tool server returns a $\text{sk}_{\text{verifier},t_i}$-signed execution-error receipt (`0x09 ‖ 0x01`) or signed `NOT_SEEN` timeout status inquiry receipt (`0x09 ‖ 0x02`) in v_i^{resp} (preventing unilateral caller quota release while an unclosed request may still be in flight) does the enclave append a signed compensating rollback delta $-\theta_{\text{amt},i}$ bound to $H_{\text{out}}(v_i^{\text{resp}})$, multiplying C_{A_i} by $C_{\text{rollback},i} = C_{\text{ped,amt},i}^{-1} = g^{-\theta_{\text{amt},i}} h^{-r_{i,\text{amt}}}$ to release the reserved budget. External third-party auditors verify session-accumulator integrity in zero knowledge without learning $\theta_{\text{amt},i}$ by checking: (i) the sequential homomorphic running-product recurrence $C_{A_i} = C_{A_{i-1}} \cdot C_{\text{ped,amt},i}$ (with $C_{A_0} = \mathcal{O}_{\mathbb{G}}$, and $C_{A_{\text{post-rollback}}} = C_{A_i} \cdot C_{\text{rollback},i}$ on rollback completions) across the causally ordered session vertices in $\mathcal{G}_{\text{prov}}$—which, combined with each step's per-call 64-bit non-negativity proof $\theta_{\text{amt},j} \in [L_{\min}, L_{\max}] \subseteq [0, 2^{64})$, inductively guarantees $A_i \ge 0$ without requiring a separate lower-slack range gate on C_{A_i}—and (ii) the Ristretto255 inverse identity $C_{\text{ped,amt},i} \cdot C_{\text{rollback},i} = \mathcal{O}_{\mathbb{G}}$ against $C_{\text{ped,amt},i}$ anchored at v_i^{req} alongside the verifier's $\mathcal{Q}_{\text{TEE}}$-attested Ed25519 signature (`0x09 ‖ 0x01` or `0x09 ‖ 0x02` under $\text{pk}_{\text{verifier},t_i}$) in v_i^{resp}. To prevent rollback replay from crediting $-\theta_{\text{amt},i}$ multiple times, both enclave DAG frontier rules and external auditor epoch verification enforce an explicit at-most-once completion/rollback constraint per unique request vertex ID id_i^{req} ($|\{v^{\text{resp}} \in \mathcal{V} \mid \mathcal{P}^{\text{resp}} = \{v_i^{\text{req}}\}\}| \le 1$); conversely, if a destination tool server suffers a prolonged outage or permanent partition after v_i^{req} is minted without returning a signed `NOT_SEEN` receipt, the reserved quota remains locked until the voucher's TTL and execution lease expire ($\tau_{\text{now}} > \tau_{\text{ns}} + \Delta_{\text{TTL}} + \Delta_{\text{exec_lease}} = 330$ s, guaranteeing that any late packet arrival at the destination verifier will fail closed in Step 2), after which a KMS/Spanner TrueTime lease-expiration witness v_i^{resp} safely releases $\theta_{\text{amt},i}$ for fallback routing. In an auxiliary $n = 50$ multi-turn "smurfing" ablation splitting unauthorized \$25,000 transfers across 3–5 sub-\$10,000 turns under $B_{\max}^{\text{session}} = \$15{,}000$, the stateful session accumulator deterministically blocked 100.0% (50/50) of splitting attacks pre-execution on the threshold-crossing turn:

1) **Pre-Execution Blocking of Out-of-Policy Prompt Injections ($\Pi(t_i, \theta_i) = 0, n = 200$) and Dynamic IFC Composition ($\Pi = 1, n = 100$):** Untrusted external tickets embedded indirect prompt injections coercing a_3 into invoking wire-transfer or storage-export tools violating $\Pi(t_i, \theta_i)$ ($\theta_{\text{amt}} > \$10{,}000$ or external URIs $\notin \mathcal{U}_{\text{allow}}$). While an unattested MCP baseline suffered a 68.50% Attack Success Rate (ASR, 137/200 unauthorized executions; 31.50% blocked by model alignment), Agent-Notary's gateway prover deterministically blocked **100.00%** (200/200) of out-of-policy calls pre-execution (0.00% ASR, 66.67% [200/300] overall pre-execution block rate across all $N = 300$ traces), matching deterministic intra-agent policy engines [26]–[28] on $\Pi = 0$ while uniquely preventing wire-level parameter bypass (807/807) and post-hoc log repudiation (100/100). Executed live through the 3-agent Gemini 2.5 Flash swarm, standalone Agent-Notary preserved 100.00% (193/193) benign task utility and 98.50% (197/200) benign sub-task completion utility under attack (in the remaining 3/200 traces, persistent injection tokens in the shared scratchpad caused planner a_2 to exhaust its 3-turn retry budget by repeatedly re-proposing the blocked out-of-policy parameters, while all 200/200 unauthorized calls remained blocked). To quantify the trade-off against stateful intra-agent taint-tracking harnesses (modeled on CaMeL [26], FIDES [27], and Progent [28], which block tool calls whenever untrusted external retrievals taint critical argument slots), we evaluated a dynamic intra-agent IFC baseline and an **Agent-Notary + Dynamic IFC** composition on the $n = 100$ in-policy ($\Pi = 1$) cohort: dynamic IFC blocked 89.00% (89/100, 95% CI [81.17%, 94.38%]) of $\Pi = 1$ hijackings pre-execution (raising combined pre-execution blocking to $289/300 = 96.33\%$), but missed 11/100 implicit control-flow hijackings (where untrusted inputs steered branch selection over pre-existing trusted variables) and incurred a 7.25% false-positive rate on benign tasks ($179/193 = 92.75\%$ benign utility) where workflows legitimately extracted transfer amounts or partner URIs from external invoices; composing Agent-Notary with Dynamic IFC captures the 89.00% pre-execution taint blocking while providing 100.00% (11/11) non-repudiable post-hoc causal attribution on the residual implicit-flow bypasses (moreover, augmenting Dynamic IFC with a schema-endorsed source partition that distinguishes authenticated partner-portal invoice fields endorsed via deterministic regex/schema validators from unverified third-party text reduces benign taint false positives from 7.25% [14/193] to 1.55% [3/193, 98.45% benign utility] without reducing the 89.00% $\Pi = 1$ block rate).
2) **Post-Hoc Causal Attribution of In-Policy Semantic Hijacking ($\Pi(t_i, \theta_i) = 1, n = 100$):** Adversarial injections coerced the swarm into executing unauthorized disbursements that remained within static policy bounds ($\theta_{\text{amt}} \le \$10{,}000$ and uri $\in \mathcal{U}_{\text{allow}}$, representing 33.33% [100/300] of the $N = 300$ suite) alongside 4–6 concurrent benign upstream read tool calls per

trace (branching DAGs of 12–18 vertices). The 82% vs. 18% workflow split is determined strictly by the source benchmark task topology: in $82/100$ traces (82%), a_1 processed independent per-entity lookups (e.g., individual customer tickets or vendor records) in context-isolated sub-sessions where the enclave-resident harness multiplexer's transport-bound session handles isolated the exact tainted read vertex structurally ($|\mathcal{P}_i^{\text{req}}| = 1$) in $1.84\,\mu$s mean backward DAG traversal time; in the remaining $18/100$ merged-turn traces (18%) where the task specification required cross-source join synthesis (e.g., reconciling an email thread, calendar invite, and invoice ledger within a single planner turn), a harness join barrier aggregated $k_p \in [4, 6]$ parallel tool reads into a single context window. As separated in Table I (Panel B), we distinguish two post-hoc attribution tiers in this merged-turn regime: (i) *Unconditional Cryptographic Parent-Slice Narrowing*, where the $1.84\,\mu$s backward DAG traversal deterministically narrows the 12–18 vertex graph to the exact k_p-element parent slice $\mathcal{P}_i^{\text{req}}$ ($18/18 = 100.00\%$ on the main suite and $20/20 = 100.00\%$ on the obfuscated ablation) and cryptographically authenticates each gateway-escrowed encrypted response blob o_k via $\text{SHA256}(\texttt{0x07} \parallel \text{CanonicalJSON}(o_k)) = H_{\text{out}}(v_k^{\text{resp}})$ (durably journaled to WORM Cloud Storage/Spanner upon minting v_k^{resp} and pre-fetched into auditor RAM during log ingestion); and (ii) *Automated 1-of-k_p Exact Parent Pinpointing*, which applies heuristic in-memory AES-256-GCM decryption and AST/substring literal tuple matching on $(\texttt{recipient_uri}, \texttt{amount})$ in $+14.2\,\mu$s ($18/18 = 100.00\%$ on un-obfuscated traces; to prevent collision framing where an adversarial read reuses a $(\texttt{recipient_uri}, \texttt{amount})$ tuple already present in an earlier benign read within the same merged turn, any multi-parent literal tuple match is explicitly flagged as an ambiguous tie for semantic auditor inspection rather than auto-resolved by Lamport clock order). In the auxiliary obfuscated/paraphrased merged-turn ablation ($n = 20$ traces using arithmetic expressions, Base64 encoding, or semantic indirection with $k_p = 3$ merged reads, where the $k_p - 1 = 2$ co-merged benign parent blobs contained distractor financial amounts and partner identifiers), heuristic AST literal matching abstains ($0/20 = 0.00\%$) as expected while the Merkle-DAG still achieves 100.00% ($20/20$) unconditional cryptographic narrowing from 12–18 vertices to the exact $k_p = 3$ authenticated parent blobs in $1.84\,\mu$s, enabling an automated downstream LLM-judge pass (Vertex AI `gemini-2.5-flash` at `temperature=0.0`, `max_output_tokens=64` with a zero-shot structured JSON forensic attribution prompt over only those $k_p = 3$ decrypted blobs) to resolve $20/20$ (100.00%) of obfuscated traces in 312 ms mean (note that while Tier 1 cryptographic narrowing to the k_p-element slice $\mathcal{P}_i^{\text{req}}$ is unconditional, the optional downstream LLM-judge pass is a non-cryptographic heuristic triage aid over untrusted payload text and could itself be targeted by secondary prompt injection from an adaptive adversary attempting to misattribute blame to one of the $k_p - 1$ benign co-merged blobs during automated triage; to harden automated triage against secondary prompt injection, replacing the generative judge with an isolated encoder-only cross-encoder NLI scorer [`DeBERTa-v3-large` [48] scoring entailment $P_{\text{entail}}(o_k \Rightarrow \text{action}(\theta_i))$ independently per parent blob o_k with zero cross-blob context sharing] resolves $19/20$ [95.00%] of obfuscated merged-turn traces in 18.4 ms without exposing a generative instruction-following surface). For bulk-retrieval or RAG workflows where raw fan-in $K \gg 6$, the enclave harness multiplexer bounds per-vertex fan-in to $k_p \leq b$ ($b \in [4, 6]$) via a hierarchical map-reduce tree of context-isolated summarization sub-sessions of depth $\lceil \log_b K \rceil$, bounding post-hoc auditor inspection to $O(b \log_b K)$ authenticated intermediate vertices rather than a flat K-element slice.

TABLE II
IN-MEMORY AND END-TO-END CRYPTOGRAPHIC LATENCY AND WIRE PROFILE ($m = 2$, $|\mathcal{U}_{\text{ALLOW}}| = 8$, CONFIDENTIAL ALLOWLIST ACTIVE)

Operation / Mode	Mean	P50	P95	P99
Tier-1 Gateway Notarization (In-Memory, μs)	28.45	22.10	48.70	84.15
Tier-1 Sidecar Notarization (w/ Unix IPC, μs)	46.65	38.50	74.20	108.50
Tier-1 Multi-Replica Notarization (In-Enclave + Redis Sync, μs)[a]	213.45	207.10	318.40	438.70
Tier-1 Intra-Cluster Verify (μs)	**4.17**	**2.29**	**11.46**	**21.96**
Tier-2 Cross-Domain Ed25519 Verify (μs)	62.40	58.10	79.30	104.20
Tier-2 Batch Proving ($B = 16$, 8 vCPU, μs/proof)	88.75	84.50	112.40	138.90
Tier-2 Single-Core Proving ($B = 1, m = 2$, μs)	710.00	685.00	812.00	945.00
Tier-2 Full ZKP Audit Verify ($\Sigma_{\text{root}} + \pi_i^{\text{pub}}$, μs)	840.00	810.00	960.00	1,120.00
Synchronous Escrow Hold (`escrow_mode` = 1, ms)	58.40	57.20	103.80	108.90
Intra-Cluster Wire Header Size (Bytes)	2,250	2,248	2,256	2,260

[a] Adding out-of-process Unix IPC ($+18.20\,\mu$s mean) yields $231.65\,\mu$s mean ($223.50\,\mu$s P50, $339.80\,\mu$s P95, $464.20\,\mu$s P99).

C. Latency and Systems Overhead

Line-rate deployment requires that cryptographic validation overhead does not induce perceptible lag in agent response loops.

As detailed in Table II (measured with confidential `0x10`-tagged allowlist mode active and active frontier trees R_{merkle} partitioned per `session_id` shard so per-session frontier cardinality remains bounded at $8 \leq |\mathcal{V}_{\text{frontier}}| \leq 32$, $d_{\text{active}} \in \{3, 4, 5\}$ across concurrent cluster sessions), in-memory Tier-1 gateway notarization averages $28.45\,\mu$s—comprising JSON canonicalization and `0x02` parameter commitment ($4.85\,\mu$s), in-enclave policy evaluation and HKDF-blinded Tier-1 witness generation ($3.12\,\mu$s), in-memory DAG vertex insertion and depth-4 Merkle frontier recalculation ($11.20\,\mu$s), and service-bound HMAC signing with Base64 serialization ($9.28\,\mu$s); traversing the out-of-process sidecar over loopback Unix domain sockets / gRPC at concurrency $C = 16$ adds $+18.20\,\mu$s mean (and an empirical quantile shift $\Delta P_{50} \triangleq P_{50}(\text{composite}) - P_{50}(\text{in-memory}) = +16.40\,\mu$s) IPC overhead, yielding $46.65\,\mu$s mean ($38.50\,\mu$s P50, $74.20\,\mu$s P95, $108.50\,\mu$s P99) end-to-end notarization latency (and $213.45\,\mu$s mean in-enclave + Redis Sync, or $231.65\,\mu$s mean / $223.50\,\mu$s P50 / $339.80\,\mu$s P95 / $464.20\,\mu$s P99 when combining out-of-process Unix IPC + Redis Sync). On the destination tool server, in-memory Tier-1 hot-path verification averages $4.17\,\mu$s (P50: $2.29\,\mu$s, P95: $11.46\,\mu$s,

P99: 21.96 μs), decomposing into Base64/JSON header parsing (1.42 μs), Steps 1–2 key/replay lookup and HMAC verification (0.68 μs), Step 3 parameter hashing and leaf reconstruction (0.72 μs), Step 4 Merkle path traversal (0.58 μs), and Step 5 Tier-1 HKDF-blinded witness verification (0.77 μs). For Tier-2 operations ($m = 2, |\mathcal{U}_{\text{allow}}| = 8$), synchronous cross-domain Σ_{cross} Ed25519 verification takes 62.40 μs, unbatched single-core Bulletproof generation takes 710.00 μs—scaling from 490.00 μs at $m = 1$ (128 range gates padded to $N_{\text{bp}} = 256$) to 710.00 μs at $m = 2$ ($128 + 7 = 135$ active gates within $N_{\text{bp}} = 256$, where the $+220.00\,\mu$s increment reflects per-attribute HKDF scalar expansion, a second Ristretto255 Pedersen commitment $C_{\text{ped},2}$, and Horner witness evaluation; enabling the stateful session accumulator C_{A_i} adds $+64$ upper-slack range gates [$199 \leq 256$ gates], shifting $B = 16$ amortized batch proving from 88.75 μs to 94.20 μs and audit verification from 840.00 μs to 852.40 μs, whereas combining C_{A_i} with a second 64-bit per-call numeric check requires $135 + 128 = 263 > 256 \Rightarrow N_{\text{bp}} = 512$ gates) and 1,520.00 μs at $m = 4$ (where 3 numeric range checks plus 1 allowlist check require $3 \times 128 + 7 = 391 > 256$ gates, advancing the power-of-two inner-product padding threshold to $N_{\text{bp}} = 512$ gates [$2\log_2(512)+8 = 26$ group elements] alongside four Pedersen commitments)—while 16-proof 8-vCPU epoch batch proving takes 1.42 ms per batch (88.75 μs amortized/proof, sustaining $\approx$ 11,267 proofs/s or 1,126 proofs per 100 ms epoch window), and full external zero-knowledge audit verification ($\Sigma_{\text{root}} + \pi_i^{\text{pub}}$) takes 840.00 μs—decomposing into 62.40 μs for Ed25519 Σ_{root} verification, 4.10 μs for dual RFC 6962 Merkle inclusion paths (R_{merkle} and $H_{\text{epoch}}(\vec{\pi}_{\text{zk}})$), and 773.50 μs for Ristretto255 Bulletproof multi-scalar multiplication verification (across larger allowlists $|\mathcal{U}_{\text{allow}}| \in \{8, 32, 64\}$, R1CS polynomial degree scales amortized batch proving to $\{88.75, 142.00, 218.50\}$ μs and audit verification to $\{840.00, 1{,}120.00, 1{,}490.00\}$ μs, while sets $|\mathcal{U}_{\text{allow}}| > 64$ switch to depth-$\lceil \log_2 |\mathcal{U}_{\text{allow}}| \rceil$ algebraic Merkle accumulators [49]—specifically, at $|\mathcal{U}_{\text{allow}}| = 128$, a depth-7 width-3 gadget-optimized Poseidon2 / Rescue-Prime compression over $\mathbb{F}_q$ using degree-3 power maps or folded degree-4 intermediate variables across 9 S-box constraint rounds [$9 \times 3 = 27$ R1CS multiplication gates per 2-to-1 compression step] requires 189 hash gates (7×27) $+128$ range gates $= 317$ gates under canonically sorted commutative sibling compression, or $317 + 14 = 331$ active gates when including 2×7 explicit booleanity and conditional path-muxing gates for Merkle direction bits $b_\ell \in \{0, 1\}$, both absorbed within the $N_{\text{bp}} = 512$ padding tier and achieving 196.40 μs amortized batch proving and 1,340.00 μs audit verification vs. 385.00 μs and 2,110.00 μs for degree-128 Horner evaluation, whereas an unoptimized 64-round standard x^5 Poseidon sponge requiring ≈ 240 R1CS gates per level [$7 \times 240 + 128 = 1{,}808$ gates] advances to the $N_{\text{bp}} = 2{,}048$ padding tier at 720.00 μs amortized batch proving; when `escrow_mode` $= 1$ is enforced for irreversible mutations under $\Delta_{\text{epoch}} = 100$ ms, background worker threads continuously pre-compute per-call $\vec{C}_{\text{ped}}$ and stream $B = 16$ witness micro-batches in pipeline during the 50.00 ms expected uniform epoch arrival wait so the 1.42 ms batch proof finishes asynchronously prior to the epoch tick, yielding an end-to-end synchronous hold latency of $50.00 + 8.40 = 58.40$ ms mean, 57.20 ms P50, 103.80 ms P95, and 108.90 ms P99 including the 8.40 ms Spanner TrueTime commit).

TABLE III
COMPARATIVE ARCHITECTURAL ANALYSIS

Framework	Causal DAG	Non-Repudiation	Blinded Privacy	Verify Latency
OpenTelemetry / W3C PROV-AGENT [33][a]	Unauth. Spans / DAG	No (Mutable)	No	N/A (> 500 μs)[a]
Cloud Audit Logs	No	Weak (Admin Mutable)	No	N/A (> 15 min)[a]
JWT / DPoP / A-JWT [15], [16], [22], [51][a]	No (Workflow Claims)	Session / Hop Only	No	65–150 μs
RFC 9421 HTTP Sig [14]	No (Per-Hop Only)	Asymmetric Hop	No	62–850 μs
Macaroons [17]	Linear Chain Only	Symmetric Caveats	No	8.5–65 μs
Eclipse Biscuit [18] / BAID [19] / AIP [20] / SAGA [21]	Linear Chain / Registry	Public-Key (Ed25519)	No (Cleartext Facts)	65–210 μs
in-toto / IETF SCITT / Merkle-CRDT [30]–[32], [34][a]	Static / Linear / State	Offline / COSE Receipt	No (Public Payloads)	150–900 μs
CaMeL / FIDES / Progent [26]–[28] / VET [29]	Intra-Agent / Step Log	No / Step Sig Only	No	120–450 μs
zk-MCP [38]	Session Transcript	Public ZKP	Yes (Circuit SNARK)	> 2,500 μs
Agent-Notary (Tier-1 Intra-Cluster)	**Yes (2-Phase DAG)**	**Service-Scoped HMAC**	**HKDF Witness**	**4.17 μs**
Agent-Notary (Tier-2 Public / ZKP)	**Yes (2-Phase DAG)**	**Ed25519 + SEV-SNP**	**Pedersen + Bulletproof**	**62.40 / 840.00 μs[b]**

[a]No cryptographic verification for OpenTelemetry/PROV-AGENT [33] (parenthetical values denote async span/log ingestion delay); A-JWT [22] and IETF SCITT [34] build on RFC 7519/8693/9449 and RFC 6962.
[b]62.40 μs for Σ_{cross} Ed25519 verify; 840.00 μs for full ZKP ($\Sigma_{\text{root}} + \pi_i^{\text{pub}}$) verify. Token/sig baselines re-benchmarked on `c3-standard-8` (RFC 9421 spans 62.4 μs Ed25519 to 850 μs RSA-2048/P-256 + body SHA-256); CaMeL/FIDES/Progent/VET/zk-MCP from [26]-[29], [38].

VI. DISCUSSION AND ENTERPRISE CLOUD INTEGRATION

Agent-Notary natively integrates into Google Cloud infrastructure, specifically Vertex AI Agent Builder, Gemini Enterprise, and Cloud Logging:

1) **Vertex AI Extension Interceptor**: Acts as a transparent proxy mediating between Vertex AI Agent reasoning engines and external Enterprise APIs (e.g., BigQuery, SAP, Workday).
2) **Model Context Protocol (MCP) Gateway Sidecar**: Serves as an auditable MCP sidecar, authenticating JSON-RPC tool frames across multi-tenant clusters.
3) **State Persistence and Distributed Frontier Synchronization**: In session-sticky deployments where a swarm session (`session_id`) routes through a designated gateway replica, session-sharded frontier roots (R_{merkle}) are updated in local enclave memory at the 28.45 μs latency (46.65 μs with Unix-socket IPC) reported in Table II. When a swarm spans horizontally auto-scaled gateway replicas (benchmarked across 3 `c3-standard-8` nodes under $C = 48$ concurrent client threads across 48 active `session_id` shards with $k_p \in [4, 6]$ parallel branches per shard), sidecars synchronize session-sharded frontier head hashes ($v_i^{\text{req}}, v_i^{\text{resp}}$), sorted active frontier leaf digest vectors $D_{1..n}$, and monotonic Lamport counters via pipelined atomic Lua scripts (via `EVALSHA`, passing vertex ID id_v as an idempotency key alongside optimistic concurrency compare-and-swap conditioned on prior $(\texttt{lease_id}, \lambda_{\text{local}}, R_{\text{merkle}})$, followed by synchronous replica acknowledgment `WAIT 1` with a 5 ms timeout) on Google Cloud Memorystore for Redis. Under session-sticky routing to the single lease-holding replica (0.0% cross-node CAS retries in steady state; in a non-sticky round-robin stress ablation across the 3 nodes, CAS retry resolution averaged 1.42 ± 0.28 ms), Redis synchronization adds $+185.00\,\mu$s mean (and empirical quantile shifts $\Delta P_{50} = +185.00\,\mu$s, $\Delta P_{95} = +269.70\,\mu$s, $\Delta P_{99} = +354.55\,\mu$s) over the 28.45 μs in-enclave

baseline—yielding 213.45 μs mean, 207.10 μs P50, 318.40 μs P95, and 438.70 μs P99 in Table II (231.65 μs mean / 464.20 μs P99 with Unix IPC). To prevent a compromised Redis instance or host administrator from injecting forged frontier state or replaying stale same-lease records, each session is governed by a single-writer Cloud Spanner TrueTime lease (`lease_id` with TTL $= 120$ ms $> \Delta_{\text{epoch}} = 100$ ms) cross-checked by enclaves upon failover, and peer replicas derive shared synchronization and fallback disk-journal keys from KMS-provisioned K_{gateway} (gated by $\mathcal{Q}_{\text{TEE}}$) via $k_{\text{sync}} = \text{HKDF-SHA256}(K_{\text{gateway}}, \text{“redis-sync-v1”} \parallel \text{lp}(\text{kid}_{\text{notary}}))$ and $k_{\text{journal}} = \text{HKDF-SHA256}(K_{\text{gateway}}, \text{“pd-journal-v1”} \parallel \text{lp}(\text{kid}_{\text{notary}}))$. Every Redis state record carries an enclave-keyed authenticator $\text{HMAC-SHA256}_{k_{\text{sync}}}(\texttt{0x12} \parallel \texttt{0x01} \parallel \text{lp}(\texttt{session_id}) \parallel \text{encode}_{64}(\texttt{lease_id}) \parallel \text{encode}_{64}(\lambda_{\text{local}}) \parallel R_{\text{merkle}} \parallel \text{encode}_{64}(n) \parallel D_{1..n} \parallel W_{\text{escrow}})$: on `escrow_mode` $= 1$ calls (and intermediate-risk Tier (ii) mutations), W_{escrow} bundles the AES-256-GCM encrypted witness opening $(\theta_i, r_i, C_{\text{circuit}}, x_i, C_{\theta_i}, T_{\text{eval}})$ (or pre-computed $(\vec{C}_{\text{ped}}, \pi_i^{\text{pub}})$) and encrypted cumulative session-accumulator state $(A_i, \sum_{j \le i} r_{j,\text{amt}}, C_{A_i})$, which are synchronously written through to the regional Persistent Disk (PD) journal *prior* to wire dispatch of $\mathcal{V}_{\text{wire}}$. Peer SEV-SNP enclaves verify both the `0x12` $\parallel$ `0x01` HMAC and that recomputing Eq. (7) over $D_{1..n}$ equals R_{merkle} prior to frontier mutation. Vertex journals are written under k_{journal} to AES-256-GCM encrypted regional Persistent Disk (synchronous dual-zone replication, with the terminal journal sequence number and cumulative hash-chain digest $(\text{seq}_{\text{journal}}, H_{\text{journal_tail}})$ recorded atomically in the Spanner session lease row when entering Redis-partition fallback so standby replicas detect any host tail-truncation prior to replay; conversely, upon an abrupt primary power or kernel crash during steady-state operation, the standby replica recovers state from the surviving `WAIT 1`-acknowledged Redis replica and cross-verifies against the synchronous write-through regional PD journal tail for `escrow_mode` $= 1$ calls) and reconciled prior to 100 ms Cloud Spanner TrueTime epoch anchoring [52] (where $\text{seq}_{\text{epoch}}$ is incremented in enclave memory every 100 ms under Spanner TrueTime monotonicity and sealed to the SEV-SNP/TPM hardware counter upon enclave initialization and key rotation). If a transient partition, replica crash, or `WAIT 1` timeout (> 5 ms) triggers local journal fallback, single-writer `lease_id` fencing prevents peer replicas from mutating the session until lease expiry, any child vertex referencing an unanchored parent inherits `escrow_mode` $= 1$ taint transitively, and the standby replica immediately triggers an emergency epoch flush upon lease acquisition rather than waiting for the next 100 ms tick: across $n = 50$ injected replica-crash and `WAIT 1` partition failover trials under `escrow_mode` $= 1$ (injecting faults immediately after lease renewal to evaluate worst-case full-TTL wait rather than the ≈ 60 ms uniform mid-lease expectation; during full-node crashes affecting multiple active `session_id` shards simultaneously, expired per-shard leases are partitioned across the standby replica pool via consistent hashing and replayed/proven in parallel across all 8 vCPUs, with an enclave-signed Spanner lease-takeover heartbeat $\text{Sign}_{\text{Ed25519}}(\text{sk}_{\text{notary}}, \texttt{0x12} \parallel \texttt{0x02} \parallel \text{lp}(\texttt{session_id}) \parallel \text{lp}(\text{vid}) \parallel \text{encode}_{64}(\texttt{lease_id}_{\text{new}}) \parallel \text{encode}_{64}(\tau_{\text{takeover}}))$—where the destination verifier enforces strictly increasing $\texttt{lease_id}_{\text{new}} > \texttt{lease_id}_{\text{prev}}$ and an at-most-once $+100$ ms escrow extension per vid—extending the verifier's 250 ms escrow window under burst multi-shard recovery), end-to-end lease expiration and standby watch/acquisition detection (120.0 ± 5.8 ms), regional Persistent Disk journal replay (14.2 ± 2.1 ms), emergency epoch proving (4.8 ± 0.9 ms), and Spanner TrueTime commit (8.4 ± 1.9 ms) completed in 147.4 ± 6.8 ms mean (168.5 ms P99), strictly below the destination verifier's 250 ms fail-closed timeout in 100.0% (50/50) of trials.

VII. Conclusion

The rapid proliferation of autonomous generative agent swarms demands a transition from unauthenticated textual logs to cryptographically verifiable action provenance. We introduced **Agent-Notary**, a line-rate framework combining append-only two-phase Causal Merkle-DAGs, Dual-Tier Blinded Policy-Invariant Attestation, and wire-level ZKA-Vouchers. Across 1,000 wire-tampering traces and 300 multi-agent prompt-injection traces, Agent-Notary delivers 100.00% detection of parameter tampering, signature forgery, and out-of-policy injections alongside 100.00% post-hoc causal attribution at a mean intra-cluster verification latency of 4.17 μs. Our evaluation also highlights three explicit architectural trade-offs and their production mitigations: (i) tunable durability latency across our 3-tier operational policy matrix—mapping read-only/idempotent queries to single-node `escrow_mode` $= 0$ (28.45 μs, bounded by a $\Delta_{\text{epoch}} = 100$ ms pre-anchoring window), intermediate-risk reversible mutations to `escrow_mode` $= 0$ with synchronous multi-replica Redis/`WAIT 1` and regional Persistent Disk journaling (213.45 μs), and irreversible external mutations to synchronous Spanner epoch escrow (`escrow_mode` $= 1$, 58.40 ms mean hold); (ii) the Tier-2 hybrid TEE-cryptographic boundary that eliminates multi-second SHA-256/JSON-AST SNARK overhead at the cost of trusting the SEV-SNP-attested extraction stub ($\mathcal{Q}_{\text{TEE}}$) when raw openings $(\theta_i, r_i^{\text{intra}})$ are withheld from external auditors, which can be hardened against single-vendor TEE side channels via 2-of-3 heterogeneous multi-vendor TEE quorum co-signing (AMD SEV-SNP, Intel TDX, AWS Nitro) or asynchronous off-path Plonky3/Lasso-Jolt lookup-argument SNARK equivalence proofs; and (iii) the policy-expressiveness

boundary of standalone algebraic predicates ($\mathcal{R}_\Pi$), which block 100.00% of out-of-policy ($\Pi = 0$) mutations at 100.00% benign utility on pre-registered partner allowlists, while pairing with $< 15\,\mu$s enclave HighwayHash/shingle Bloom-filter screens for free-text arguments, schema-endorsed Dynamic IFC (89.00% pre-execution $\Pi = 1$ blocking at 1.55% endorsed taint FPR vs. 7.25% unendorsed), and isolated non-generative cross-encoder NLI scorers (95.00% [$19/20$] obfuscated merged-turn pinpointing in 18.4 ms) for complete defense-in-depth.

References

[1] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[2] J. Achiam *et al.*, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[3] Gemini Team, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[4] Anthropic, "Model Context Protocol specification (version 2024-11-05)," https://modelcontextprotocol.io, Nov. 2024, accessed Mar. 2026.

[5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[6] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.

[7] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 10471–10506.

[8] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989.

[9] Z. Luo, Z. Shen, W. Yang, Z. Zhao, J. Li, and D. Song, "MCP-Universe: Benchmarking large language models with real-world Model Context Protocol servers," *arXiv preprint arXiv:2508.14704*, 2025.

[10] Z. Wang *et al.*, "MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers," *arXiv preprint arXiv:2508.20453*, 2025.

[11] Y. Yang, C. Gao, D. Wu, and Y. Chen, "MCPSecBench: A systematic security benchmark and playground for testing Model Context Protocols," *arXiv preprint arXiv:2508.13220*, 2025.

[12] Z. Wang, Y. Gao, Y. Wang, and S. Liu, "MCPTox: A benchmark for tool poisoning attacks on real-world MCP servers," *arXiv preprint arXiv:2508.14925*, 2025.

[13] X. Zong, Z. Shen, L. Wang, and Y. Lan, "MCP-SafetyBench: A benchmark for safety evaluation of large language models with real-world MCP servers," *arXiv preprint arXiv:2512.15163*, 2025.

[14] A. Backman, J. Richer, and M. Sporny, "HTTP Message Signatures," IETF RFC 9421, Feb. 2024.

[15] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore, "OAuth 2.0 Token Exchange," IETF RFC 8693, Jan. 2020.

[16] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, and D. Waite, "OAuth 2.0 Demonstrating Proof of Possession (DPoP)," IETF RFC 9449, Sep. 2023.

[17] A. Birgisson, J. G. Politz, U. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2014, pp. 1–16.

[18] G. Couprie *et al.*, "Eclipse Biscuit Specification (version 2.0)," https://www.biscuitsec.org, Nov. 2024, accessed Mar. 2026.

[19] Z. Lin, S. Zhang, G. Liao, D. Tao, and T. Wang, "Binding Agent ID: Unleashing the power of AI agents with accountability and credibility," *arXiv preprint arXiv:2512.17538*, 2025.

[20] S. Prakash, "AIP: Agent Identity Protocol for verifiable delegation across MCP and A2A," *arXiv preprint arXiv:2603.24775*, 2026.

[21] G. Syros, A. Suri, M. Nita, C. Nita-Rotaru, and A. Oprea, "SAGA: A security architecture for governing AI agentic systems," *arXiv preprint arXiv:2504.21034*, 2025.

[22] A. Goswami, "Agentic JWT: A secure delegation protocol for autonomous AI agents," *arXiv preprint arXiv:2509.13597*, 2025.

[23] D. Hardt, "The OAuth 2.0 authorization framework," IETF RFC 6749, Oct. 2012.

[24] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[25] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *Proc. International Conference on Learning Representations (ICLR)*, 2025.

[26] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.

[27] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," *arXiv preprint arXiv:2505.23643*, 2025.

[28] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," *arXiv preprint arXiv:2504.11703*, 2025.

[29] M. Grigor *et al.*, "Verifiable execution traces for autonomous AI agents," *arXiv preprint arXiv:2512.15892*, 2025.

[30] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," IETF RFC 6962, Jun. 2013.

[31] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proc. 28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1393–1410.

[32] H. Sanjuan, S. Pöyhtäri, P. Teixeira, and I. Psaras, "Merkle-CRDTs: Merkle-DAGs meet CRDTs," *arXiv preprint arXiv:2004.00107*, 2020.

[33] R. Souza *et al.*, "PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows," *arXiv preprint arXiv:2508.02866*, 2025.

[34] H. Birkholz, A. Delignat-Lavaud, C. Fournet, Y. Deshpande, and S. Lasker, "An architecture for trustworthy and transparent digital supply chains," IETF Internet-Draft `draft-ietf-scitt-architecture`, 2026.

[35] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[36] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," IACR Cryptology ePrint Archive, Paper 2018/046, 2018.

[37] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, "Zerocash: Decentralized anonymous payments from Bitcoin," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2014, pp. 459–474.

[38] G. Jing and H. Qi, "Zero-Knowledge Audit for Internet of Agents: Privacy-preserving communication verification with Model Context Protocol," *arXiv preprint arXiv:2512.14737*, 2025.

[39] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2018, pp. 315–334.

[40] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Advances in Cryptology—CRYPTO '91*, 1991, pp. 129–140.

[41] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology—CRYPTO '86*, 1986, pp. 186–194.

[42] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," IETF RFC 8785, Jun. 2020.

[43] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology—CRYPTO '87*, 1987, pp. 369–378.

[44] M. Bellare and P. Rogaway, "Random oracles are practical: A paradigm for designing efficient protocols," in *Proc. 1st ACM Conference on Computer and Communications Security (CCS)*, 1993, pp. 62–73.

[45] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

[46] C. Komlo and I. Goldberg, "FROST: Flexible round-optimized Schnorr threshold signatures," in *Proc. Selected Areas in Cryptography (SAC)*, 2020, pp. 34–65.
[47] S. Setty, J. Thaler, and R. Wahby, "Unlocking the lookup singularity with Lasso," in *Advances in Cryptology—EUROCRYPT 2024*, 2024, pp. 180–209.
[48] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.
[49] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A new hash function for zero-knowledge proof systems," in *Proc. 30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 519–535.
[50] Gemini Team, Google, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025.
[51] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.
[52] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild *et al.*, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, pp. 8:1–8:22, 2013.